Final Year Project Documentation

Lost and Found

(Centralized Platform for Persons and Items)

By

Farwa Izhar	**076920**
Adeeba Khan	**076992**
Kiran Shehzadi	**076996**

Project ID: **BSIT-22-G26**

Under the Supervision of

Prof. Muhammad Asif Afzal

Bachelor of Science in IT (2022-2026)

Department of Computer Science & Information Technology

Govt. Graduate Islamia College, Gujranwala

Lost and Found: For Lost and Found Persons and Items

A project presented to

University of the Punjab, Lahore

In partial fulfillment
of the requirement for the degree of

Bachelors of Science in IT (2022-2026)

By

Farwa Izhar	076920
Adeeba Khan	076992
Kiran Shehzadi	076996

Department of Computer Science & Information Technology,

Govt. Graduate Islamia College, Gujranwala

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Farwa Izhar
Signature: _____

Adeeba Khan
Signature: _____

Kiran Shehzadi
Signature: _____

CERTIFICATE OF APPROVAL

It is to certify that the final year design project (FYDP) of BSIT "Lost and Found System" was developed by **Farwa Izhar, Kiran Shehzadi, and Adeeba khan** under the supervision of "**Prof. Muhammad Asif Afzal**" in my opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in IT.

FYDP Supervisor: Prof. Muhammad Asif Afzal

Signature: ----------

Signatures (Faculty Advisory Committee (FAC)

	FAC1	FAC2	FAC3
Names			
Signature			

Head of FYDP Coordination Office:

Signature: ----------

Dated: _____

Chairperson, Department of [BSIT]

Signature: ----------

Executive Summary

The *Lost and Found* is a web-based application designed to help people report, search, and track missing or found persons and lost or found items. The system provides a centralized platform where users can easily post reports with details and images, search using filters, and interact through comments and sharing features.

The main purpose of this project is to solve the problem of scattered and unorganized information when someone loses a person or item. Currently, people mostly rely on social media or personal communication, which leads to delays, confusion, and lack of proper tracking. This system overcomes these issues by offering a structured and reliable solution.

The application allows both registered and unregistered users to search and view reports, while registered users can create posts, comment, and interact with others. An admin plays an important role by verifying reports, managing inappropriate content, updating case status, and sending alerts when a match is found between lost and found reports.

The system is developed using modern technologies such as the MERN stack (MongoDB, Express.js, React.js, and Node.js), ensuring scalability, flexibility, and efficient data handling. It also includes features like image uploading, secure authentication, and statistical analysis to improve user experience.

In conclusion, the *Lost and Found* system is a user-friendly and efficient solution that helps individuals and communities reconnect lost persons and items with their rightful owners by providing a reliable, fast, and organized digital platform.

Acknowledgement

First of all, we are thankful to **Almighty Allah** for blessing us with strength, patience, and guidance to complete this project successfully.

We extend our deepest and heartfelt gratitude to our esteemed supervisor, **Prof. Muhammad Asif**, for his valuable guidance, continuous support, and helpful suggestions during this project. His knowledge, encouragement, and supervision were the key reasons we were able to improve our work and complete it successfully.

We are also thankful to the Head of Department and the respected faculty members of **BS(IT) at Islamia Graduate College, Gujranwala,** for providing a positive academic environment and the opportunity to enhance our knowledge and skills.

Furthermore, we acknowledge the dedicated efforts of our project group members, **Farwa Izhar**, **Kiran**, and **Adeeba Khan**, whose cooperation, hard work, and teamwork played an important role in the successful completion of this project.

Farwa Izhar

Signature: _____

Adeeba Khan

Signature: _____

Kiran Shehzadi

Signature: _____

Abbreviations

All the abbreviations and acronyms used in our FYP along with their respective brief description.

Abbreviation	Full Form	Description
API	Application Programming Interface	A set of protocol that allow different software systems to communicate.
CSS	Cascading Style Sheet	A style sheet language used for describing the presentation of a web page.
DB	Database	A structured collection of data stored efficiently.
HTML	Hyper Text Markup Language	The standard markup language for creating web pages.
JS	JavaScript	A programming language used for client-side scripting on web.
MongoDB	Mongo Database	A database that stores data in JSON-like documents.
Node.js	Node JavaScript	A runtime environment for executing JavaScript on the server side.
React	React.Js	A JavaScript library building fast and dynamic user interfaces.
UI	User Interface	The part of a system with which users interact.

Table of Contents

Chapter 1 ..... **17**
Introduction 17
1. Introduction 18
1.1 Problem Statements 18
1.2 Problem Solutions 18
1.3 Objectives of the Proposed System 19
1.4 Scope 19
1.10 Project Deliverables 19
1.5 System Components 19
1.5.1 Module 1: Report Module 19
1.5.2 Module 2: Statistics Module 20
1.5.3 Module 3: Authentication Module 20
1.5.4 Module 4: Admin Dashboard 20
1.5.5 Module 5: Notification Module 20
1.6 Related System Analysis / Literature Review 20
1.7 Vision Statement 20
1.8 System Limitations and Constraints 20
1.9 Tools and Technologies 21
1.9 Work Division 22
1.11 Project Planning 22
1.12 Summary 22
Chapter 2 ..... **23**
Requirement Analysis 23
2. Analysis 24
2.1 User classes and characteristics 24
2.2 Requirement identifying techniques 24
2.2.1 Observation Technique 24
2.2.2 Literature Review 25
2.3 Functional Requirements 25
2.3.1 Functional Requirement 1 25
2.3.2 Functional Requirement 2 26
2.3.3 Functional Requirement 3 26
2.3.4 Functional Requirement 4 27
2.3.5 Functional Requirement 5 27
2.3.6 Functional Requirement 6 28

2.3.7 Functional Requirement 7 28
2.3.8 Functional Requirement 8 29
2.3.9 Functional Requirement 9 29
2.3.10 Functional Requirement 10 30
2.3.11 Functional Requirement 11 30
2.3.12 Functional Requirement 12 31
2.4 Non-Functional Requirements 31
2.5 Use Case Analysis 32
2.5.1 Use Case 1 32
2.5.2 Use Case 2 33
2.5.3 Use Case 3 34
2.5.4 Use Case 4 35
2.5.5 Use Case 5 36
2.5.6 Use Case 6 37
2.5.7 Use Case 7 38
2.5.8 Use Case 8 38
2.5.9 Use Case 9 39
2.5.10 Use Case 10 40
2.5.11 Use Case 11 41
2.5.12 Use Case 12 42
2.5.13 Use Case 13 43
2.5.14 Use Case 14 44
2.5.15 Use Case 15 45
2.5.16 Use Case 16 46
2.5.17 Use Case 17 47
2.5.18 Use Case 18 47
2.5.19 Use Case 19 48
2.5.20 Use Case Diagram 49
2.6 External Interface Requirements 50
2.6.1 User Interface Requirements 50
2.6.2 Software Interfaces 50
2.6.3 Hardware Interfaces 50
2.6.4 Communication Interfaces 50
Chapter 3 51
Design and Architecture 51
3.System Design 52

3.1 Design Considerations 52
3.1.1 Assumptions: 52
3.1.2 Dependencies: 52
3.1.3 Limitations: 52
3.1.4 Risks: 52
3.2 Design Models 52
3.2.1 Use Case Diagram 52
3.2.2 Class Diagram 53
3.3 Architectural Design 54
3.3.1 UML Component Diagram 54
3.3.2 MVC Model 56
3.4 Data Design 57
3.4.1 Data Dictionary 58
3.5 User Interface Design 62
3.5.1 Screen Images 62
3.6 Behavioral Model 67
3.6.1 Sequence Diagram 67
3.6.2 State Transition Diagram 71
3.7 Design Decisions 72
3.8 Summary 72
Chapter 4 73
Implementation 73
4. Implementation 74
4.1 Algorithm 74
4.2 External APIs 82
4.3 Code Repository 82
4.4 Summary 82
Chapter 5 83
Testing and Evaluation 83
Testing and Evaluation 84
5.1 Unit Testing (UT) 84
5.1.1 All Unit Test Cases (UT) Brief Specifications 85
5.1.2 Test Case 1 85
5.1.3 Test Case 2 86
5.1.4 Test Case 3 86
5.1.5 Test Case 4 87

5.1.6 Test Case 5 88
5.1.7 Test Case 6 89
5.1.8 Test Case 7 89
5.1.9 Test Case 8 90
5.1.10 Test Case 9 91
5.1.11 Test Case 10 91
5.2 Functional Testing (FT) 92
5.2.1 Functional Test Cases (FT) Brief Description 92
5.2.2 Test Case 1 93
5.2.3 Test Case 2 94
5.2.4 Test Case 3 94
5.2.5 Test Case 4 95
5.2.6 Test Case 5 96
5.2.7 Test Case 6 96
5.3 Integration Test Cases (IT) 97
5.3.1 Test Case 97
5.3.2 Test Case 2 98
5.3.3 Test Case 3 98
5.3.4 Test Case 4 99
5.3.5 Test Case 5 99
5.4 Performance Testing 100
5.4.1 Test Case 1 100
5.4.1 Test Case 2 101
5.4.1 Test Case 3 101
5.3.4 Test Case 4 102
5.5 Summary 102
Chapter 6 103
System Conversion 103
6. Introduction 104
6.1 Conversion Method 104
6.2 Deployment 104
6.2.1 Data Conversion 105
6.2.2 Training 105
6.3 Post Deployment Testing 105
6.4 Challenges 106
6.5 Summary 106

Chapter 7 ..... **107**
Conclusion 107
7. Introduction 108
7.1 Evaluation 108
7.2 Traceability Matrix 109
7.3 Conclusion 109
7.4 Future Work 110
References **110**
Appendix A **111**
Use Case Description Template 111
UC-1: User Registration 112
UC-2: User Login 112
UC-3: Create Report 113
UC-4: Search & Filter Reports 114
UC-4: Comment on report 114
UC-6: View Statistics 115
UC-7: View Posts 115
UC-8: Share Posts 116
UC-9: View Comments 117
UC-10: Edit Reports 117
UC-11: Delete Report 118
UC-12: Flag Content 118
UC-13: Reset Password 119
UC-14: Log Out 120
UC-15: Manage Users 120
UC-16: Verify Cases by Tracking 121
UC-17: Update Case Status 122
UC-18: Manage Posts 122
UC-19: Send Alerts to Users 123
Appendix-B **124**
Coding Standards 124
General Coding Standards & Guidelines 125
1.Consistent Variable Naming 125
2. Meaningful Naming 125
3.Immutable Fields 125
4.Controlled Access 125

5.Function Naming 125
6.Comments Usage 125
7.Indentation & Formatting 125
8.File & Folder Naming 125
9.Class Structure 125
10. Code Organization 125
11.Minimum Scope Usage 125
12.Access Modifiers 125
13.High Cohesion & Low Coupling 125
14.DRY Principle 125
15.Single Responsibility Principle 126
16.Extenable Code 126
17.Exception Handling 126
18.Logging 126
19.Consistent Logging Style 126
Appendix C 127
Prototypes 127

List of Figures

Figure 1: Use Case Diagram 49
Figure 2: Use Case Diagram 53
Figure 3: Class Diagram 54
Figure 4: UML Component Diagram 55
Figure 5: MVC Model 56
Figure 6: ER Diagram 66
Figure 7: Sequence Diagram 1 67
Figure 8: Sequence Diagram 2 68
Figure 9: Sequence Diagram 3 69
Figure 10: Sequence Diagram 4 70
Figure 11: Sequence Diagram 5 70
Figure 12: State Transition Diagram 71

List of Tables

Table 1: Tools, Libraries and Technologies 21
Table 2: User Class and Characteristics for Proposed Project 24
Table 3: Description of FR-1 25
Table 4: Description of FR-2 26
Table 5: Description of FR-3 26
Table 6: Description of FR-4 27
Table 7: Description of FR-5 27
Table 8: Description of FR-6 28
Table 9: Description of FR-7 28
Table 10: Description of FR-8 29
Table 11: Description of FR-9 29
Table 12: Description of FR-10 30
Table 13: Description of FR-11 30
Table 14: Description of FR-12 31
Table 15: Non-Functional Requirements 31
Table 16: Use Case 1 32
Table 17: Use Case 2 33
Table 18: Use Case 3 34
Table 19: Use Case 4 35
Table 20: Use Case 5 36
Table 21: Use Case 6 37
Table 22: Use Case 7 38
Table 23: Use Case 8 38
Table 24: Use Case 9 39
Table 25: Use Case 10 40
Table 26: Use Case 11 41
Table 27: Use Case 12 42
Table 28: Use Case 13 43
Table 29: Use Case 14 44
Table 30: Use Case 15 45
Table 31: Use Case 16 46
Table 32: Use Case 17 47
Table 33: Use Case 18 47
Table 34: Use Case 19 48
Table 35: Algorithm 1 74
Table 36: Algorithm 2 75
Table 37: Algorithm 3 75
Table 38: Algorithm 4 76
Table 39: Algorithm 5 77
Table 40: Algorithm 6 78
Table 41: Algorithm 7 78
Table 42: Algorithm 8 79
Table 43: Algorithm 9 80
Table 44: Algorithm 10 80
Table 45: Algorithm 11 81

Table 46: External APIs 82
Table 47: All Unit Test Cases 85
Table 48: All functional Test Cases 92
Table 49: mapping of BRs with actual implementation 108
Table 50: Traceability Matrix 109

Chapter 1

Introduction

1. Introduction

This chapter gives an overview of the project. It explains why the system is needed and what problems it solves. It also describes the main goals of the system. This chapter helps us to understand the purpose of the project. It also shows how the system will support users in real life.

1.1 Problem Statements

1) Many people face problems when their personal items get lost. In some cases, even people go missing. There is no proper system where such cases can be reported easily. Most people use social media to share information. These platforms are not organized. Information gets lost very quickly. It becomes hard to track old posts.
2) There is also no proper way to verify the information. Many fake or incomplete posts are shared. This creates confusion for users. People cannot trust all the information available online. There is also no proper search system. Users cannot filter data based on location or time.
3) Another problem is the lack of communication. People who post and people who find items cannot easily connect. There is also no alert system when a match is found. Many existing apps only focus on either items or persons. They do not provide a complete solution.
4) There is also no proper statistical data available. Users cannot see reports. This makes it hard to understand the situation of lost items or people. Due to all these problems, the available systems are slow and difficult to recover lost items or people. Therefore, there is a need for a proper system.

1.2 Problem Solutions

1) To solve these problems, a web-based system is proposed. This system will provide a centralized platform. Users can report missing or found people and items. The system will allow users to upload images and details. This will help to better identification of lost items or people.
2) Users will be able to search and filter reports easily. They can search by location, type, or date. The system will also provide searches from stored data. These statistics will show monthly data of reports. This will help users understand trends in the system.
3) The system will include a secure login feature. Users can register and manage their reports. They can also comment on posts and share them. A reporting system will be available to report fake posts.
4) An admin will manage the system. The admin will verify all reports. This will reduce fake information. The admin will also match lost and found reports. When a match is found, alerts will be sent to users.
5) The system will also include an optional FIR field. This will improve trust in reports. Notifications will be sent through email. Overall, the system will improve communication and speed up recovery of lost items/people.

1.3 Objectives of the Proposed System

1) Allow users to report lost or found items.
2) Provide search results with filters.
3) Reduce fake reports by 70% using admin verification.
4) Send alert notifications after a match is confirmed.
5) Support at least 500 users at the same time without system delay.
6) Allow users to upload images.
7) Display monthly statistics with 100% accurate data.
8) Provide secure login with encrypted passwords for all users.
9) Enable users to reset passwords using email OTP.
10) Ensure system availability of 95% uptime.

1.4 Scope

The system will be a web-based platform. It will be accessible through browsers. Users can search and view reports without registration. Registered users can create and manage reports.

The system includes reporting of lost and found persons and items. It allows image upload and optional FIR details. Users can comment and share posts. Admin will verify all reports.

The system will include search and filter features. It will also provide statistics and charts. Notifications will be sent on matching cases.

Stakeholders include users, admin, and community members. The system does not include GPS tracking. It is not directly connected to police or government systems.

The project will be completed in phases. First phase includes planning and design. Second phase includes development. Third phase includes testing and deployment.

1.10 Project Deliverables

- Project proposal
- SRS document
- Design document
- Working web application
- Final project report
- Presentation slides

1.5 System Components

1.5.1 Module 1: Report Module

- Create lost or found reports
- Upload images
- Edit or delete reports
- Search reports by filters
- Add FIR details (optional)
- ID card picture (required)

1.5.2 Module 2: Statistics Module

- Apply filters (location, type, date)
- View statistics in charts

1.5.3 Module 3: Authentication Module

- User registration
- Login and logout
- Password reset using email OTP
- Role-based access

1.5.4 Module 4: Admin Dashboard

- Verify reports
- Delete fake reports
- Match lost and found cases
- Update case status

1.5.5 Module 5: Notification Module

- Send alerts on matching
- Send email notifications
- Notify users about updates

1.6 Related System Analysis / Literature Review

Application Name	**Weakness**	**Proposed Project Solution**
Zainab Alert	Only for persons. No statistics. No alerts.	Supports both persons and items with alerts and statistics
Zara Alert	Many bugs. Limited features.	Stable system with full features and verification
Lostings	Only for items. Paid features.	Free system with full functionality
LostnFoundPak	Limited features and no matching system	Provides matching system with admin verification

1.7 Vision Statement

This system is specially designed for people who lose items or loved ones and need a reliable solution. The Lost and Found system is a web-based platform that helps users report, search, and recover lost persons and items. Unlike social media platforms, our system provides a centralized, verified, and organized system with matching and alert features.

1.8 System Limitations and Constraints

Limitations

- No live GPS tracking
- Not connected to government systems
- Depends on user-provided data

Constraints

- Limited development time
- Limited team resources
- Use of MERN stack only
- Internet connection required

1.9 Tools and Technologies

***Table 1*: Tools, Libraries and Technologies**

	Tools	**Version**	**Rationale**
	VS Code	1.89	Lightweight
	Libraries	**Version**	**Rationale**
	Express	4.19.2	APIs routing
	mongoose	8.6.1	MongoDB connection + schema
	JWT	9.0.2	Login authentication
	Bycrypt.js	2.4.3	Password hashing
	cors	2.8.5	Allow frontend & backend communication
	Multer	2.0.0	Image upload on server
Tools, Libraries & Technologies	Chart.js	V4	Show data visually
	nodemailer	6.x	for sending emails
	Axios	1.6.x	For sending and receiving HTTPS requests between frontend and backend
	Technologies	**Version**	**Rationale**
	HTML, CSS	Latest	For responsive and structured
	React.js	18.3.1	Interactive frontend UI
	Node.js	20.x	Handles requests and response
	Express.js	4.18.2	Backend API development
	MongoDB	7.0	For storing data

1.9 Work Division

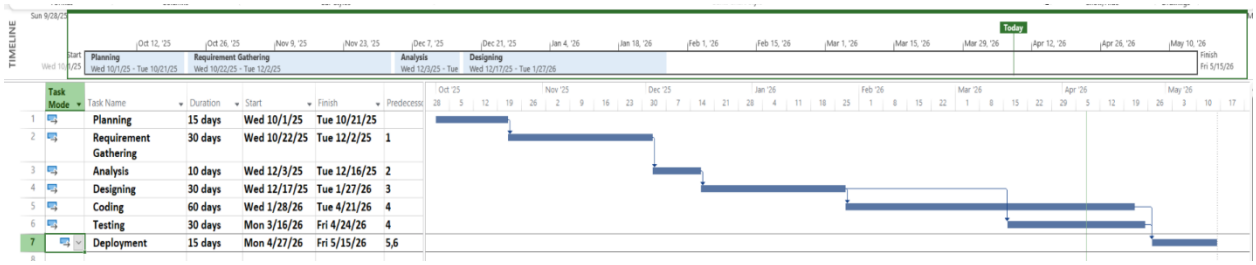
Sr. No	Roll Numbers	Names	Roll Assignment
1.	076920	Farwa Izhar	Frontend+documentation
2.	076996	Kiran Shehzadi	Backend+Documentation
3.	076992	Adeeba Khan	Database+Testing+documentation

1.11 Project Planning

The project will follow these phases:

- Planning
- Requirement gathering
- System design
- Development
- Testing
- Deployment

Figure 1: Gant chart



1.12 Summary

This chapter explained the overview of the project. It described the problem and its solution. It also explained the objectives and scope of the system. System components and technologies were discussed. Related systems were also analyzed. This chapter helps in understanding the purpose and direction of the project.

Chapter 2

Requirement Analysis

2. Analysis

This chapter explains how the requirements of the **Lost and Found** System are collected, analyzed, and organized. It describes the types of users, techniques used to gather requirements, and classification of requirements into functional and non-functional.

The main purpose of this chapter is to clearly define what the system will do and how it will behave.

2.1 User classes and characteristics

These are the user classes and their characteristics are as follows:

Table 2: User Class and Characteristics for Proposed Project

User Class	**Characteristics**
Unregistered Users	Can view, search, and filter reports. Can also view statistics and share posts. Cannot post or comment without registration.
Registered User	Can create lost/found reports, upload images, comment on posts, share reports, report on inappropriate posts, delete/update own reports and can update case status.
Admin	Has full control of the system. Can verify reports, delete fake or inappropriate posts, update case status, and send alerts on matching.

2.2 Requirement identifying techniques

The requirements of the **Lost and Found** system are identified using multiple techniques to ensure that all user needs and system functionalities are properly captured. The following techniques are used:

2.2.1 Observation Technique

Observation technique is used to understand real-world problems faced by users. It was observed that people usually rely on social media or personal contacts to find lost items or persons, which leads to confusion and delays.

Based on this observation, the following requirements were identified:

- Need for a centralized system
- Easy reporting and searching

- Quick communication through notifications

2.2.2 Literature Review

Literature review is used to study existing systems and applications related to lost and found services. Existing platforms such as lost item tracking systems and missing person applications were analyzed.

From this analysis, the following gaps were identified:

- Lack of a combined system for both persons and items
- Limited search and filtering features
- No proper alert system for matching reports
- Lack of statistics and data visualization

These gaps helped in identifying the requirements that are included in the proposed system.

2.3 Functional Requirements

Functional requirements describe what the system should do.

2.3.1 Functional Requirement 1

Table 3: Description of FR-1

Identifier	FR-1
Title	User Registration
Requirement	The user shall be able to register by entering name, email, phone and password.
Source	Business Requirement
Rationale	To allow users to create account and access system features.
Business Rule	Email must be unique and password must be strong.
Dependencies	Internet connection, Database
Priority	High

2.3.2 Functional Requirement 2

Table 4: Description of FR-2

Identifier	FR-2
Title	User Login
Requirement	The user shall be able to login using email and password.
Source	Business Requirement
Rationale	To provide secure access to the system.
Business Rule	Only registered users can login.
Dependencies	FR-1
Priority	High

2.3.3 Functional Requirement 3

Table 5: Description of FR-3

Identifier	FR-3
Title	Create Report
Requirement	The user shall be able to create a report for lost or found item/person by entering details and uploading image.
Source	Business Requirement
Rationale	To allow users to report missing or found cases.
Business Rule	Report must include title, description and essential information.
Dependencies	FR-2
Priority	High

2.3.4 Functional Requirement 4

Table 6: Description of FR-4

Identifier	FR-4
Title	Search and Filter Reports
Requirement	The user shall be able to search and filter reports based on keywords, city and type.
Source	Business Requirement
Rationale	To help users quickly find relevant reports.
Business Rule	Filters must return accurate results.
Dependencies	Database
Priority	High

2.3.5 Functional Requirement 5

Table 7: Description of FR-5

Identifier	FR-5
Title	Update/Delete Own Report
Requirement	The user shall be able to update or delete their own reports.
Source	Business Requirement
Rationale	To allow users to manage their reports.
Business Rule	User can only modify their own reports.
Dependencies	FR-3
Priority	High

2.3.6 Functional Requirement 6

Table 8: Description of FR-6

Identifier	FR-6
Title	Comment on Report
Requirement	The user shall be able to add comments on reports.
Source	Business Requirement
Rationale	To allow communication between users.
Business Rule	Comment must not be empty.
Dependencies	FR-2
Priority	Medium

2.3.7 Functional Requirement 7

Table 9: Description of FR-7

Identifier	FR-7
Title	Report on Post (Flag Content)
Requirement	The user shall be able to report inappropriate or fake posts.
Source	Business Requirement
Rationale	To maintain system quality and prevent misuse.
Business Rule	Multiple reports can mark a post as suspicious.
Dependencies	FR-2
Priority	Medium

2.3.8 Functional Requirement 8

Table 10: Description of FR-8

Identifier	FR-8
Title	Matching Detection
Requirement	The system shall detect possible matches between lost and found reports.
Source	Business Requirement
Rationale	To help users find their lost items or persons quickly.
Business Rule	Matching is based on similarity of data (location, description).
Dependencies	FR-3
Priority	High

2.3.9 Functional Requirement 9

Table 11: Description of FR-9

Identifier	FR-9
Title	Admin Verification
Requirement	The admin shall be able to verify or reject reports.
Source	Business Requirement
Rationale	To ensure authenticity of reports.
Business Rule	Only admin can verify reports.
Dependencies	Admin Module
Priority	High

2.3.10 Functional Requirement 10

Table 12: Description of FR-10

Identifier	FR-10
Title	Notification System
Requirement	The system shall send notifications for matches, comments, and updates.
Source	Business Requirement
Rationale	To keep users updated about their reports.
Business Rule	Notifications must be sent to correct user.
Dependencies	FR-8
Priority	High

2.3.11 Functional Requirement 11

Table 13: Description of FR-11

Identifier	FR-11
Title	Reset Password
Requirement	The user shall be able to reset password via email.
Source	Business Requirement
Rationale	To recover user account access.
Business Rule	Reset link must be time-limited.
Dependencies	FR-1
Priority	Medium

2.3.12 Functional Requirement 12

Table 14: Description of FR-12

Identifier	FR-12
Title	Statistics / Analysis
Requirement	The user shall be able to view statistics and analysis of reports.
Source	Business Requirement
Rationale	To provide insights about lost/found trends.
Business Rule	Data must be accurate and updated.
Dependencies	Database
Priority	Medium

2.4 Non-Functional Requirements

These requirements do not describe what the system does, but how well the system performs its functions. They include performance, security, usability, reliability, and other system qualities. These requirements apply to the system as a whole and ensure that the system is efficient, secure, and user-friendly.

Following are the non-functional requirements for the Lost and Found System:

Table 15: Non-Functional Requirements

Identifier	Category	Requirement Description	Module	Priority
NFR-1	Performance	The system should load all pages and data within 2–3 seconds to ensure fast user experience.	All Modules	High
NFR-2	Security	The system should securely store user data and passwords using encryption and authentication mechanisms.	Authentication Module	High
NFR-3	Usability	The system should provide a simple, user-friendly interface so that users can easily navigate and use features without training.	Frontend Module	Medium

NFR-4	Compatibility	The system should work properly on different devices and browsers including mobile, tablet, and desktop.	All Modules	High
NFR-5	Scalability	The system should handle increasing number of users and reports without affecting performance.	Backend Module	Medium
NFR-6	Reliability	The system should operate continuously without crashes and ensure that all data is saved correctly.	All Modules	High

2.5 Use Case Analysis

Use case analysis shows how different users interact with the system. Each use case explains a specific action performed by the user.

2.5.1 Use Case 1

Table 16: Use Case 1

Use Case ID	UC-1
Use Case Name	User Registration
Actors	Unregistered User
Description	This use case allows a new user to create an account in the system.
Trigger	User clicks on "Sign Up" button
Preconditions	User must not already be registered
Postconditions	User account is created successfully
Normal Flow	1. User opens system 2. Clicks on Sign Up 3. Enters name, email, password, phone 4. System validates data 5. Account is created

Alternative Flows	Email already exists → show error Invalid data → ask user to correct
Business Rules	Email must be unique and valid
Assumptions	User has internet connection

2.5.2 Use Case 2

Table 17: Use Case 2

Use Case ID	UC-2
Use Case Name	User Login
Actors	Registered User
Description	This use case allows a user to login into the system
Trigger	User clicks on "Login"
Preconditions	User must be registered
Postconditions	User is redirected to dashboard
Normal Flow	1. User enters email and password 2. System verifies credentials 3. User logged in successfully
Alternative Flows	Wrong password → error message User not found
Business Rules	Authentication must be secure
Assumptions	System server is active

2.5.3 Use Case 3

Table 18: Use Case 3

Use Case ID	UC-3
Use Case Name	Create Report (Lost/Found)
Actors	Registered User
Description	User can create a report for lost or found item/person
Trigger	User clicks "Create Report"
Preconditions	User must be logged in
Postconditions	Report is saved in database
Normal Flow	1. User opens report form 2. Enters title, description, location etc. 3. Uploads image 4. Submits report
Alternative Flows	Missing fields → show error Image upload fails
Business Rules	Report must contain required details
Assumptions	User provides correct information

2.5.4 Use Case 4

Table 19*: *Use Case 4

Use Case ID	UC-4
Use Case Name	Search and Filter Reports
Actors	All Users
Description	Users can search and filter reports
Trigger	User enters search query
Preconditions	Reports must exist in system
Postconditions	Filtered results are shown
Normal Flow	1. User enters keyword 2. Applies filters (location/type/name) 3. System displays results
Alternative Flows	No results found
Business Rules	Search must be fast and accurate
Assumptions	Database is available

2.5.5 Use Case 5

Table 20*: *Use Case 5

Use Case ID	UC-5
Use Case Name	Comment on Report
Actors	Registered User
Description	User can comment on reports
Trigger	User clicks "Comment"
Preconditions	User must be logged in
Postconditions	Comment is added to report
Normal Flow	1. User writes comment 2. Clicks submit 3. Comment saved
Alternative Flows	Empty comment → error
Business Rules	Comment must not be empty
Assumptions	User behaves appropriately

2.5.6 Use Case 6

Table 21*: *Use Case 6

Use Case ID	UC-6
Use Case Name	View Statistics
Actors	All Users
Description	User views system statistics.
Trigger	Open stats page
Preconditions	Data exists
Postconditions	Stats displayed
Normal Flow	1. Open page 2. View stats
Alternative Flows	No data
Business Rules	Data must be correct
Assumptions	System active

2.5.7 Use Case 7

Table 22*: *Use Case 7

Use Case ID	UC-7
Use Case Name	View Posts
Actors	All Users
Description	User views posts in system.
Trigger	Open homepage
Preconditions	Posts exist
Postconditions	Posts displayed
Normal Flow	1. Open system 2. View posts
Alternative Flows	No posts
Business Rules	Only valid posts shown
Assumptions	Data available

2.5.8 Use Case 8

Table 23*: *Use Case 8

Use Case ID	UC-8
Use Case Name	Share Posts
Actors	All Users
Description	User shares posts with others.

Trigger	Click share
Preconditions	Post exists
Postconditions	Post shared
Normal Flow	1. Select post 2. Click share
Alternative Flows	Share fails
Business Rules	Share allowed
Assumptions	Internet available

2.5.9 Use Case 9

Table 24*: *Use Case 9

Use Case ID	UC-9
Use Case Name	View Comments
Actors	All Users
Description	User views comments on posts.
Trigger	Open comments
Preconditions	Comments exist
Postconditions	Comments shown
Normal Flow	1. Open post 2. View comments

Alternative Flows	No comments
Business Rules	Comments visible
Assumptions	Data available

2.5.10 Use Case 10

Table 25*: *Use Case 10

Use Case ID	UC-10
Use Case Name	Edit Report
Actors	Registered User
Description	User edits report.
Trigger	Click edit
Preconditions	Owner access
Postconditions	Updated report
Normal Flow	1. Edit 2. Save
Alternative Flows	Unauthorized
Business Rules	Only owner
Assumptions	Logged in

2.5.11 Use Case 11

Table 26*: *Use Case 11

Use Case ID	UC-11
Use Case Name	Delete Report
Actors	Registered User
Description	User deletes report.
Trigger	Click delete
Preconditions	Owner
Postconditions	Deleted
Normal Flow	1. Select delete 2. Confirm
Alternative Flows	Cancel
Business Rules	Only owner delete
Assumptions	User confirmed

2.5.12 Use Case 12

Table 27*: *Use Case 12

Use Case ID	UC-12
Use Case Name	Flag Content
Actors	Registered User
Description	User flags inappropriate content.
Trigger	Click flag option
Preconditions	Logged in
Postconditions	Content flagged
Normal Flow	1. Select post 2. Click flag 3. Submit
Alternative Flows	Duplicate flag
Business Rules	Valid reason required
Assumptions	Honest reporting

2.5.13 Use Case 13

Table 28*: *Use Case 13

Use Case ID	UC-13
Use Case Name	Reset Password
Actors	Registered User
Description	User resets password using email.
Trigger	Click forgot password
Preconditions	User registered
Postconditions	Password updated
Normal Flow	1. Enter email 2. Receive link 3. Set new password
Alternative Flows	Invalid email
Business Rules	Link expires
Assumptions	Email service working

2.5.14 Use Case 14

Table 29*: *Use Case 14

Use Case ID	UC-14
Use Case Name	Log Out
Actors	Registered User
Description	User logs out from system.
Trigger	Click logout
Preconditions	User logged in
Postconditions	Session ended
Normal Flow	1. Click logout 2. Session cleared
Alternative Flows	Session expired
Business Rules	Session must end
Assumptions	User active

2.5.15 Use Case 15

Table 30*: *Use Case 15

Use Case ID	UC-15
Use Case Name	Manage Users
Actors	Admin
Description	Admin manages user accounts.
Trigger	Admin opens dashboard
Preconditions	Admin logged in
Postconditions	Users managed
Normal Flow	1. View users 2. Add/update/delete
Alternative Flows	Invalid action
Business Rules	Only admin allowed
Assumptions	Admin authorized

2.5.16 Use Case 16

Table 31*: *Use Case 16

Use Case ID	UC-16
Use Case Name	Verify Cases by Tracking
Actors	Admin
Description	Admin verifies cases using details.
Trigger	Admin selects case
Preconditions	Case exists
Postconditions	Case verified
Normal Flow	1. Open case 2. Check 3. Verify
Alternative Flows	Invalid data
Business Rules	Only verified cases shown
Assumptions	Accurate data

2.5.17 Use Case 17

Table 32: Use Case 17

Use Case ID	UC-17
Use Case Name	Update Case Status
Actors	Registered users
Description	Admin updates case status.
Trigger	Select case
Preconditions	Case exists
Postconditions	Status updated
Normal Flow	1. Select case 2. Update status 3. Save
Alternative Flows	Invalid status
Business Rules	Status must be valid
Assumptions	Admin active

2.5.18 Use Case 18

Table 33: Use Case 18

Use Case ID	UC-18
Use Case Name	Manage Posts
Actors	Admin
Description	Admin manages system posts.
Trigger	Admin selects post
Preconditions	Posts exist

Postconditions	Post updated/deleted
Normal Flow	1. View posts 2. delete flagged or inappropriate contents
Alternative Flows	Unauthorized
Business Rules	Admin control only
Assumptions	Admin logged in

2.5.19 Use Case 19

Table 34*: *Use Case 19

Use Case ID	UC-19
Use Case Name	Send Alerts to Users
Actors	Admin
Description	Admin sends alerts/notifications to users.
Trigger	Admin sends alert
Preconditions	Users exist
Postconditions	Alert delivered
Normal Flow	1. Create alert 2. Send to users
Alternative Flows	Delivery fails
Business Rules	Correct users notified
Assumptions	Notification system working

2.5.20 Use Case Diagram

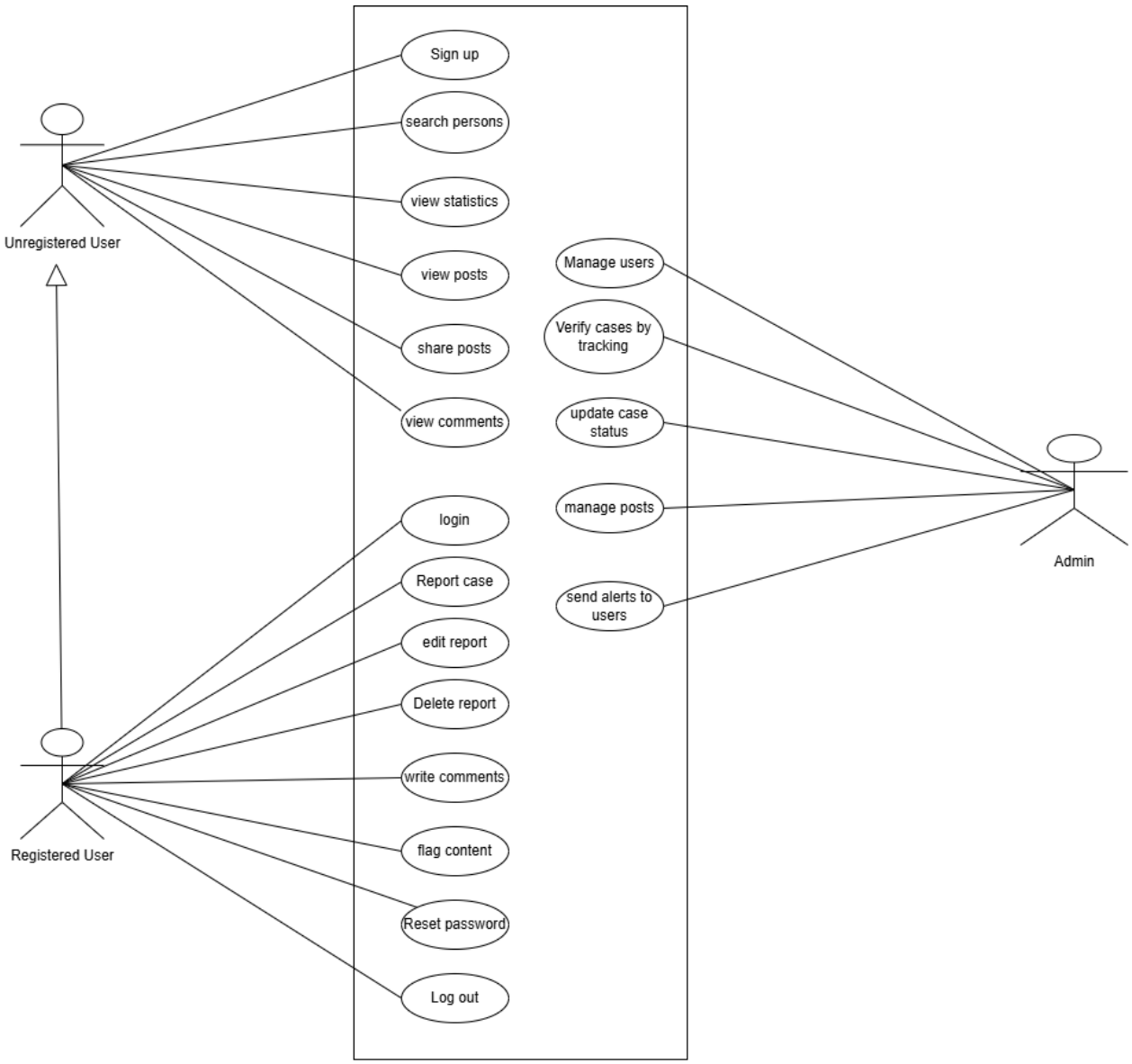


Figure 1: Use Case Diagram

2.6 External Interface Requirements

External interface requirements explain how the system interacts with users, other software, hardware devices, and communication networks. These interfaces help the system to work properly and allow users to access different features easily.

2.6.1 User Interface Requirements

The Lost and Found System provides a simple and easy-to-use user interface. Users can access the system through a web browser. The interface includes different pages such as Sign Up, Login, Home Page, Persons, Items, Stats, Search, and Dashboard. All buttons, forms, and menus are clearly designed so that users can easily understand how to use the system.

The system is responsive, which means it can work on different devices like mobile phones, tablets, and computers. The layout adjusts automatically according to screen size. The design is kept simple so that even non-technical users can use the system without difficulty.

2.6.2 Software Interfaces

The system interacts with different software components to perform its functions. It uses a web browser such as Google Chrome or Microsoft Edge for user interaction. The frontend of the system is developed using modern web technologies, while the backend is built using Node.js and Express.js.

MongoDB is used as the database to store user data, reports, and other information. The system also uses APIs to connect the frontend with the backend. In addition, email services are used for features like password reset and sending notifications to users.

2.6.3 Hardware Interfaces

The Lost and Found System does not require any special hardware devices. It can run on standard devices such as laptops, desktop computers, and smartphones. Users only need a device that supports a web browser and has an internet connection. On the server side, a system with sufficient memory and processing power is required to handle multiple users at the same time. However, no additional hardware equipment is needed for normal system usage.

2.6.4 Communication Interfaces

The system uses internet communication to connect users with the server. It works using standard protocols such as HTTP and HTTPS. HTTPS is used to ensure secure communication and protect user data. The system also uses communication services like email to send important information to users. For example, users receive emails for password reset, alerts, or important updates. All communication between the client and server is handled smoothly to ensure fast and secure data transfer.

Chapter 3

Design and Architecture

3.System Design

Product Perspective:

The lost and found is a web-based platform that works in a browser and connects to an online server and database. The software runs on the internet and allows users to interact through frontend for browsing and uploading reports, while the backend will manage authentication, data storage and system logic.

3.1 Design Considerations

3.1.1 Assumptions:

- User must have stable internet connection.
- User will provide true and accurate information.
- User have basic knowledge (for filling the form).

3.1.2 Dependencies:

- Frontend (React Native): The frontend depends on the backend APIs.
- Backend (Node.js+ Express): It depends on database.
- Database (Mongo DB): Stores all application data.
- Authentication System: The system relies on JWT and Bcrypt

3.1.3 Limitations:

- Not connected to Government Systems.

3.1.4 Risks:

- Fake Reports.
- Users may upload unclear or low quality image.
- User may drop inappropriate comment.

3.2 Design Models

3.2.1 Use Case Diagram

Description: The Use Case diagram shows how different users interact with the System. It explains what actions each type of user can perform inside the system.

System involves three main actors: Unregistered User, a Registered User and an Admin.

Unregistered User: An Unregistered User can perform basic actions like viewing, searching, and sharing posts or comments, and viewing statistics.

Registered User: A Registered User can do all of the above plus they can add or remove posts about items or persons that are missing or found.

Admin: Admin can manage all users, verify cases, update case status, manage posts and on matching sends alerts to all users.

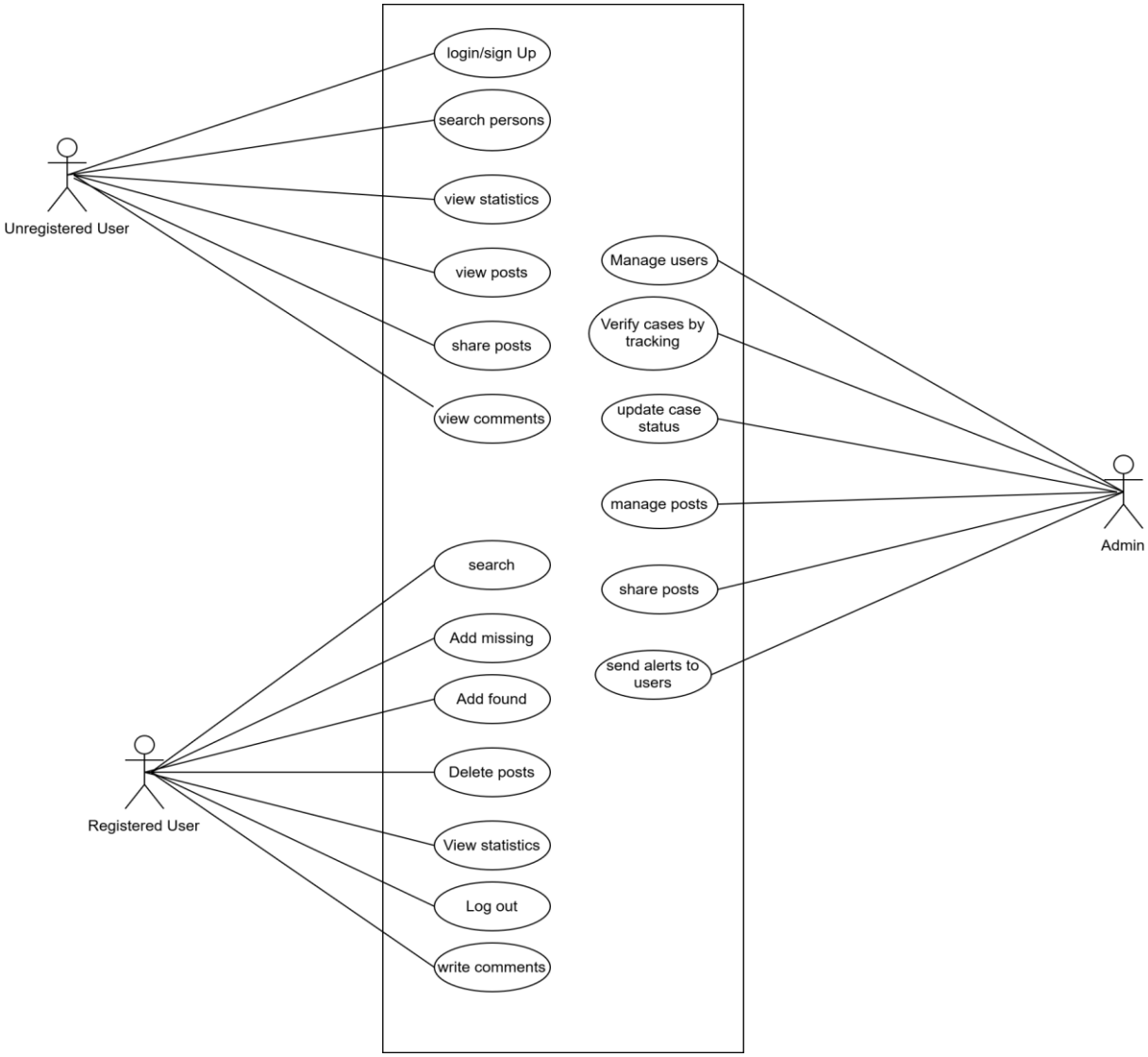


Figure 2: Use Case Diagram

3.2.2 Class Diagram

Description: The Class Diagram represents the static structure of the system by showing the system's classes, their attributes and relationship. It defines the core entities like User, Admin, Report, Report match, Alert, Comment, Image, and Statistics**.**

User: User is the parent class with common attributes like, id name, email, and password. User can be registered or not registered.

Report: User upload a report about missing and found person or item.

Admin: Admin can verify, case status update and sends alert on matching.

Comment: User can also comment on the post.

Alert: If the report match admin will send alert to the user.

Statistics: User can search, filter and view statistics.

Image: User can upload image.

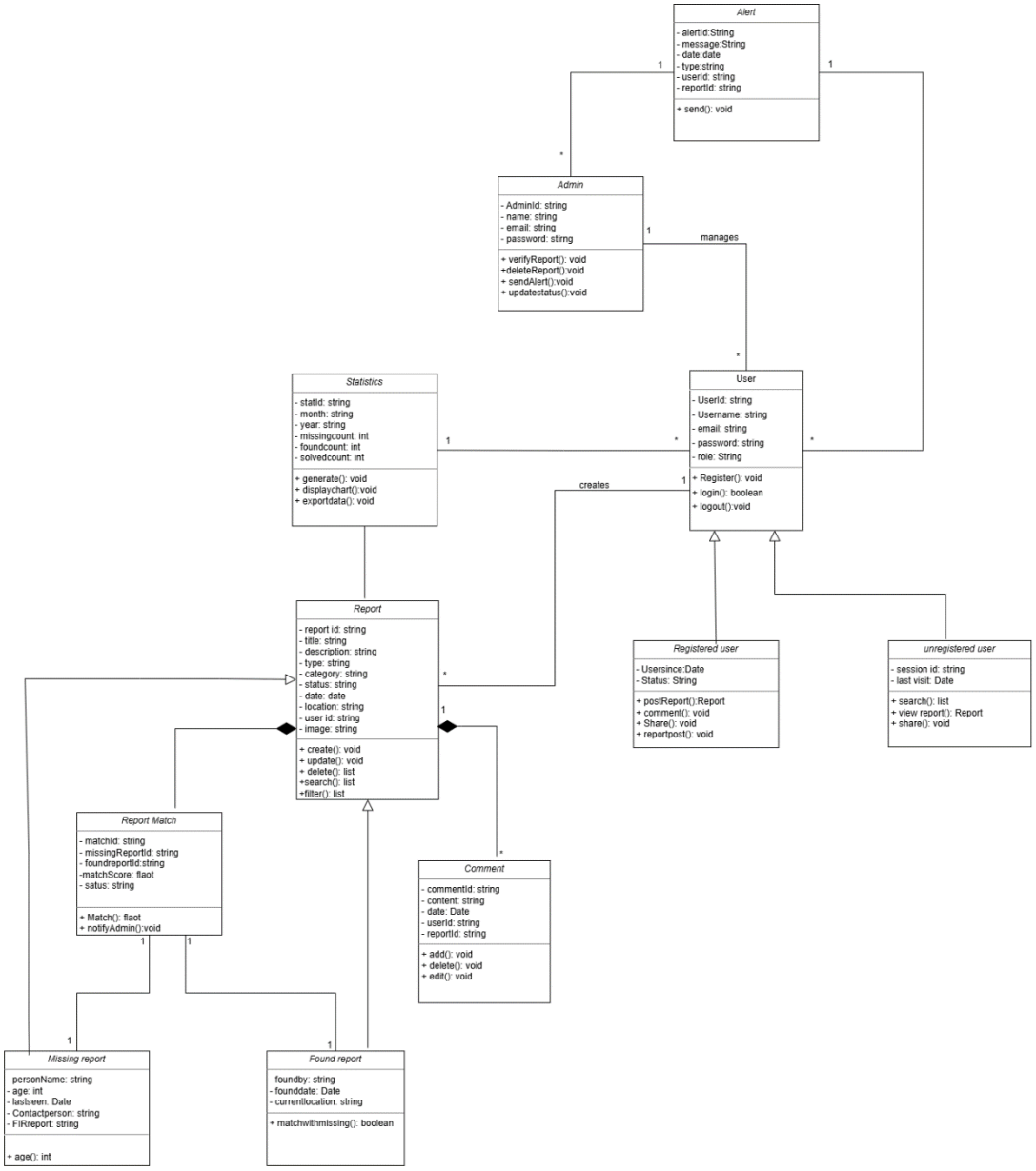


Figure 3: Class Diagram

3.3 Architectural Design

3.3.1 UML Component Diagram

Description: A UML Component is like a ready-made building block of a system that does a specific job such as handling logins or generating reports. Every Frontend (user side) module is connected to its Backend (Server Side) component. Admin side also uses these same components to control and monitor the system. All data is stored in a Database (MongoDB).

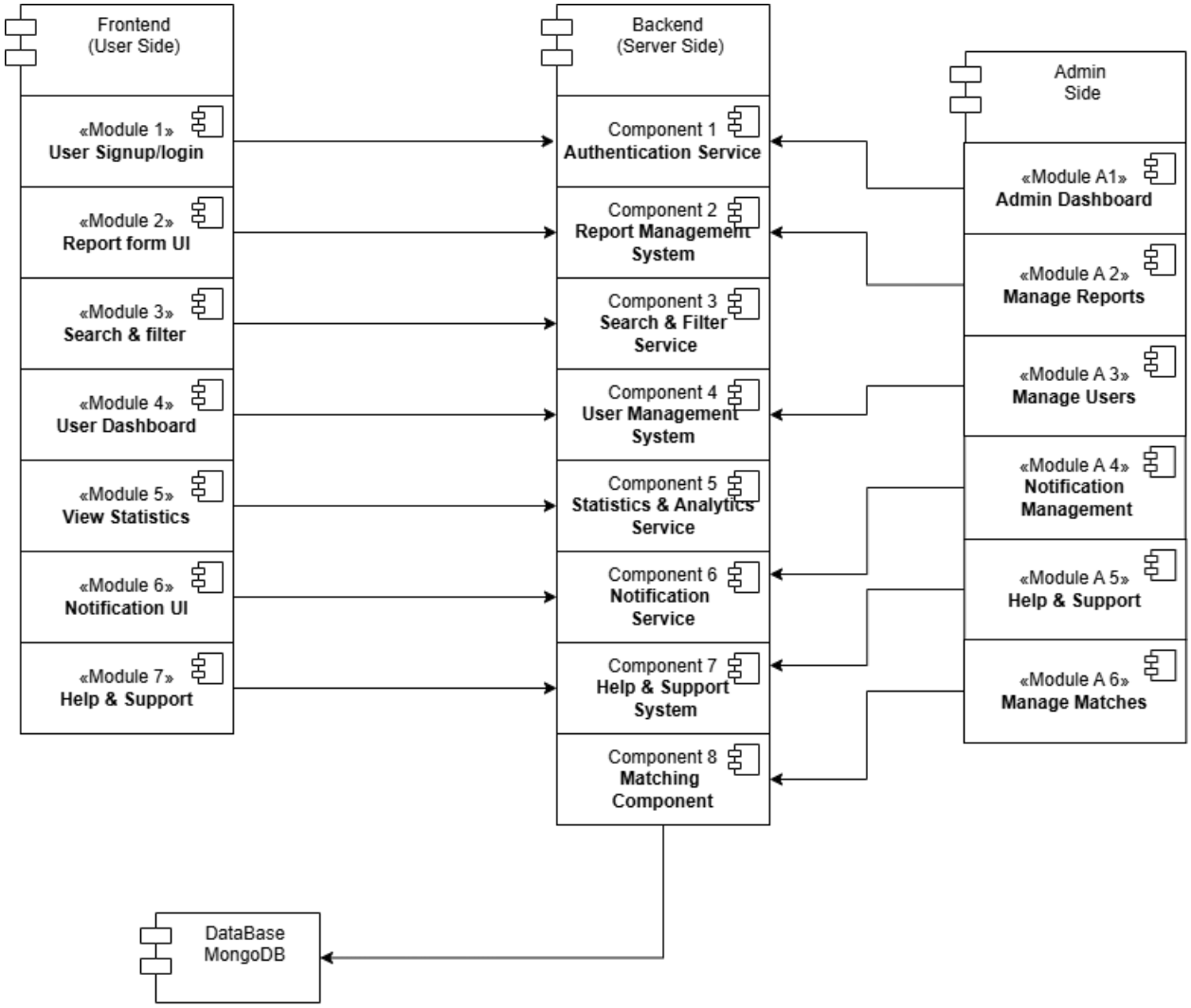


Figure 4: UML Component Diagram

3.3.2 MVC Model

Description: The MVC architectural pattern is a way to organize a system. It shows the flow of data and control between its three main components. The **Model** handles the data like reports and user information. The **View** shows this data to users, and the **Controller** decides what to do when users interact with the system.

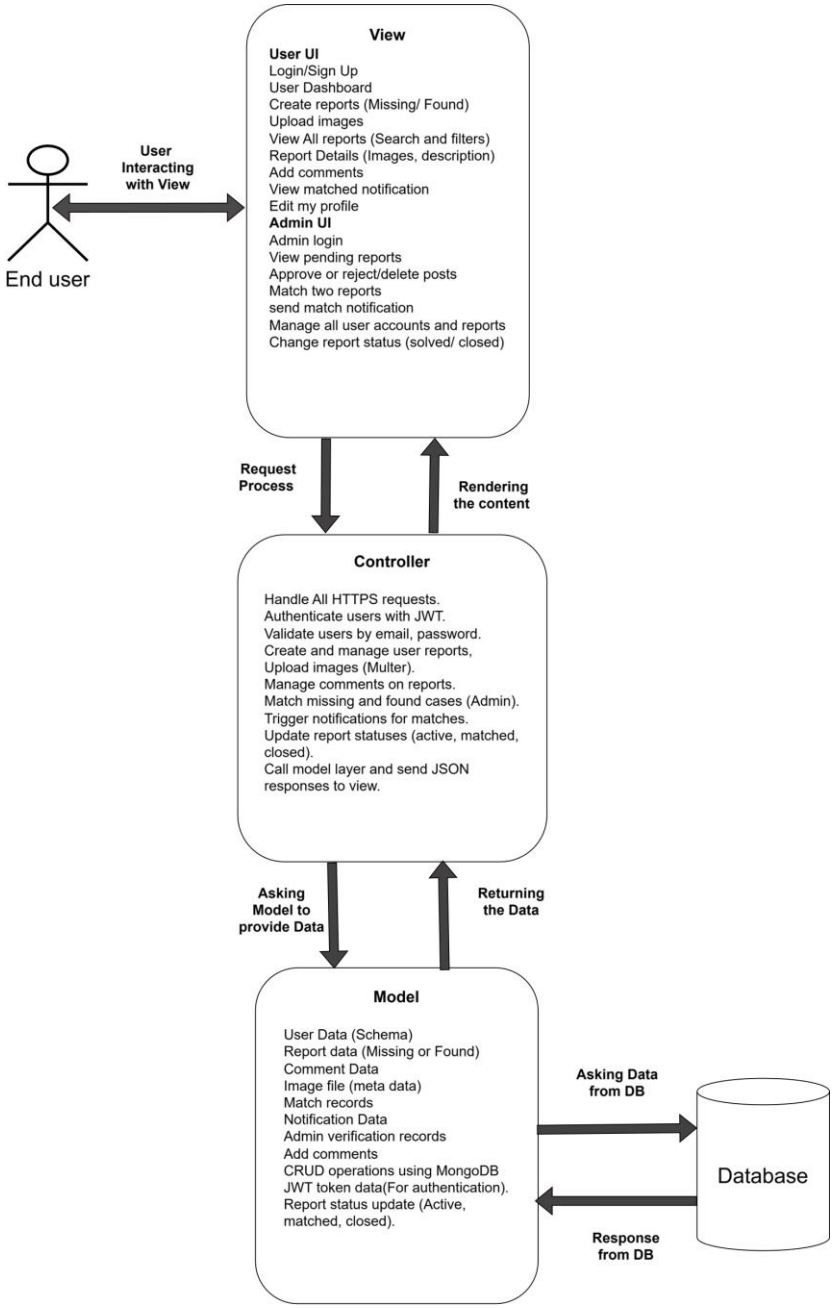


Figure 5: MVC Model

3.4 Data Design

Data Design is the process of deciding how all the information in our system will be stored, organized, and processed so the system can work effectively.

Information Domain System

The information domain system consists of these entities:

Major Entities

1. **User:** Represents a person using the system (can be admin or regular user). Stores information like name, email, password, contact and role.
2. **Report:** Represents a lost and found items or persons submitted by a user. Stores information like item/person name, description, date, location and status.
3. **Comments:** Stores feedback/comments made by users on reports.
4. **Image/File:** Stores photos or files attached to reports.
5. **Notifications:** Alerts users about updates, matches or comments related to their reports.
6. **Matching Report:** Represents the system's attempt to match the lost and found cases.

Key Relationship

- One User can create many reports (One to Many).
- A Report can have many comments (One to Many).
- One report can have multiple images/files (One to Many).
- User can receive many notifications (One to Many).
- A Report generates many notifications (One to Many).
- User can write many comments (One to Many).
- One lost report can match with multiple found reports and one found report can also match with multiple lost reports (Many to Many).

Data Storage

The Database is designed using MongoDB, which stores data in JSON-like documents inside collections. Each collection represents a major entity of our system. Related Data is linked through Object IDs. Arrays and Embedded documents are used for flexible and efficient storage. Data is stored in a flexible, scalable pattern to support fast searching, matching and updates.

Data Processing

The system performs operations like:

Create: User can create lost or found reports, user can comment on reports, system generate notifications for matches, system creates matching records for possible report matches.

Read: User can search and filter reports based on location and category, user can view comments and notifications, admin can view all reports for verification.

Update: User can update their reports, their profile and admin can update report status.

Delete: User can delete their post and comment admin can remove fake reports or inappropriate comments.

3.4.1 Data Dictionary

Alphabetical List of System Entities or Major Data with Types and Descriptions

Entities and Data

1. **Comment**
    - Comment_ id (Object id): Unique ID for each comment
    - Report _id (Object id): Links the comment to a lost/found report
    - User_ id (Object id): Shows who posted the comment.
    - Text (String): The message written by the user.
    - Created_ at (Date): Time when the comment was added.

    Purpose: Allows users to communicate or share details related to a post.

2. **Image/File**
    - File_ id (Object id): Unique ID for each image or file.
    - Report_ id (Object id): Links image/file to a report.
    - File_ URL (String): Location/path of the uploaded file.
    - File_ type (String): PNG, JPG, PDF etc.
    - Uploaded_ at (Date): When the file was uploaded.

    Purpose: Stores pictures or documents that help identify the lost/found item or person.
3. **Matching Report**
    - Match_ id (Object id): Unique id for each matching report.
    - Lost_ Report_ id (Object id): ID of the lost item or person report.
    - Found_ Report_ Id (Object id): ID of the found item or person report.
    - Matched _ By (Object id): Admin who confirmed match.
    - Matched_ At (Object id): Time of match status.

    Purpose: Store connections between lost and found reports for auto/manual matching.
4. **Notification**
    - Notification_ id (Object id): Unique id for each notification.
    - User_ id (Object id): Who will receive the notification.
    - Type (String): like match, comment, status change.
    - Message (String): Notification text.
    - Is_ Read (Boolean): Tracks whether user has opened it.
    - Created_ At (Date): Time of notification.

    Purpose: Alerts users about updates, matches, or comments.
5. **Report**

- Report_ id (Object id): Unique id for each report.
- User_ id (Object id): The user who created the report.
- Type (String): lost or found.
- Title (String): Short title describing item or person.
- Description (String): Details of the incident.
- Location (String): Where it was lost or found.
- Images (Array of Strings): List of file URLs.
- Status (String): Open/Matched/Closed.
- Created _At (Date): Posted time.

Purpose: Main Information Storage about lost or found items and persons.

6. **User**
    - User_ id (Object id): Unique user identification.
    - Name (String): Full Name.
    - Email (String): User email.
    - Phone (String): Contact Number.
    - Password_ hash (String): Encrypted password.
    - Role (String): User/Admin.
    - Created_ At (Date): Signup Time.

    Purpose: Stores account details for authentication and access.

Terminology	**Description**
User	Represents an account in the system. Stores personal information, login details, and role.
Report	Represents a lost and found persons or items submitted by a user.
Comment	Represents user feedback or questions on a report.
Image/File	Represents the pictures or documents uploaded with a report.
Notifications	A notification alerts the user about important updates.
Matching Report	A match connects a lost report with a found report when they look similar.

Structured Approach: Functions and Function Parameters

The functions and function parameters are given below:

1.Create User (): Adds a new user to the system.

Parameters:

- Name (String): Users full name.
- Email (String): Users email for login.
- Phone (String): Users contact number.
- Password (String): Users password.

2.Login User (): Checks the users email and password to log in.

Parameters:

- Email (String): Users registered email.
- Password (String): Users login password.

3.Create Report (): Adds a new lost or found report.

Parameters:

- User_ id (Object id): Who created the report.
- Type (String): Type of Lost or found.
- Description (String): Details about the item or person.
- Location (String): Where it was lost or found.
- Images (Array of strings): Pictures of the item or person.

4. Update Report Status (): Changes the status of a report like open, matched or closed.

Parameters:

- Report_ id (Object id): Which Report to update.
- Status (String): New Status

5.Add Comment (): Adds a comment to a report.

Parameters:

- Report_ id (Object id): Which report the comment belongs to.
- User_ id (Object id): Who is adding the comment.
- Text (String): Comment content.

6.Upload File (): Adds a picture or file to a report.

Parameters:

- Report_ id (Object id): Which report the file or picture belongs to.
- File (File): Image or document.

7.Generate Match (): Finds possible matches between lost and found reports.

Parameters:

- Lost_ Report_ id (Object id): lost report
- Found_ Report_ id (Object id): found report.

8.Send Notification (): Sends a message to users about matches, comments, or status update.

Parameters:

- User_ id (Object id): Who receives the notification.
- Message (String): Text of the notification.
- Type (String): Match, comment, status change.

9.View Report (): Shows the details of a report.

Parameters:

- Report_ id (Object id): Which report to view.

10.Get User Reports (): Gets all reports created by a user.

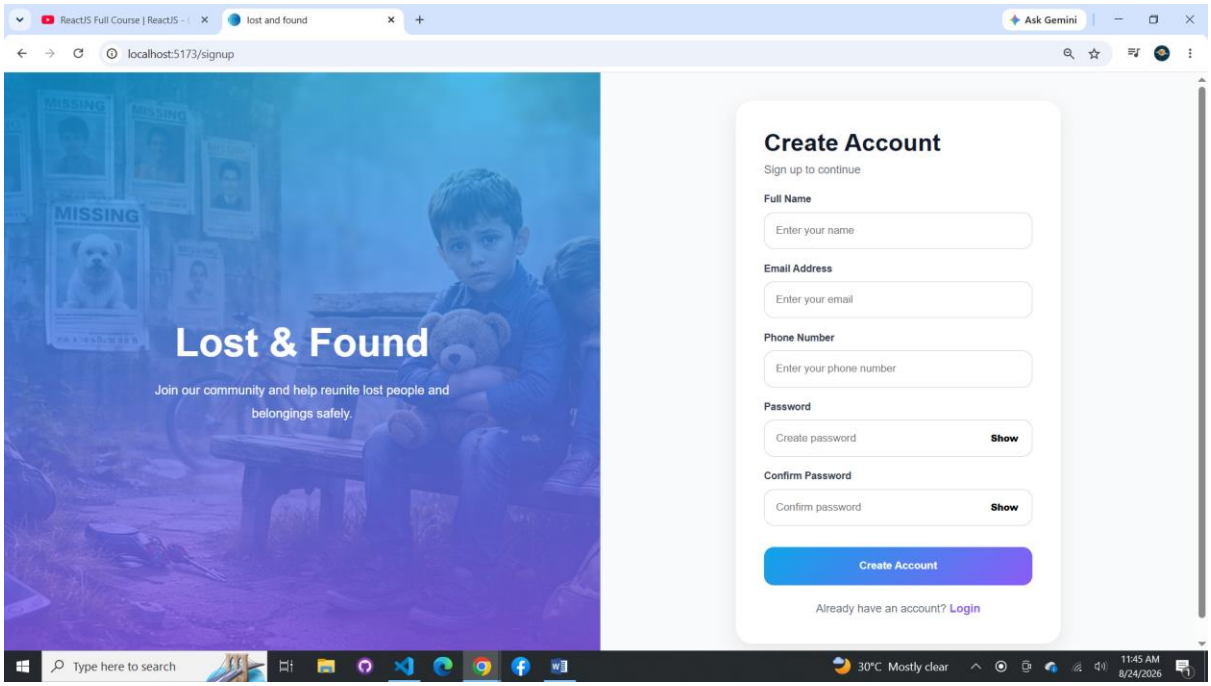
Parameters:

- User_ id (Object id): User whose reports you want.

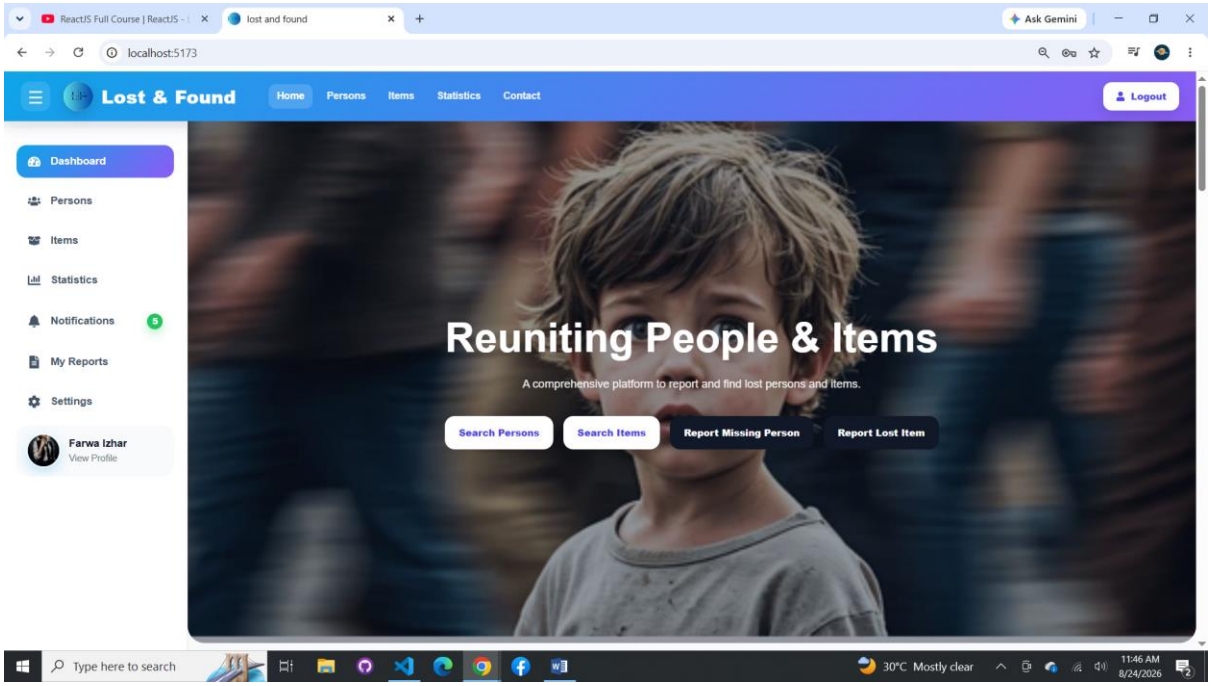
3.5 User Interface Design

3.5.1 Screen Images

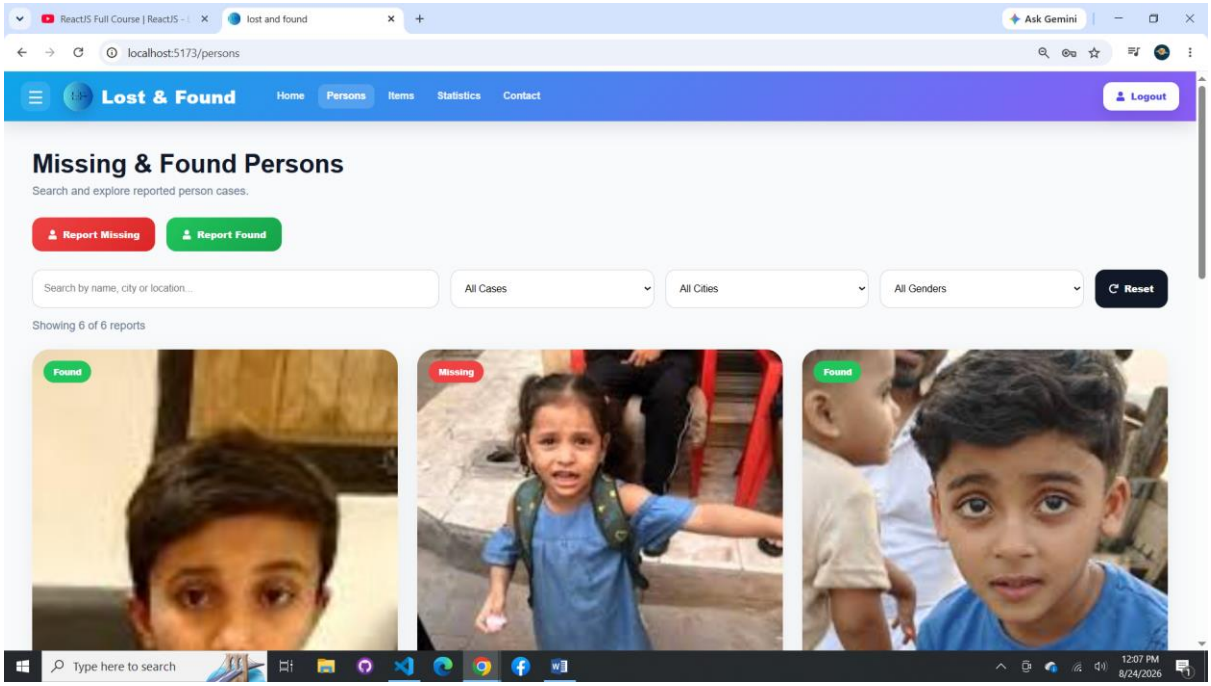
Sign Up Page



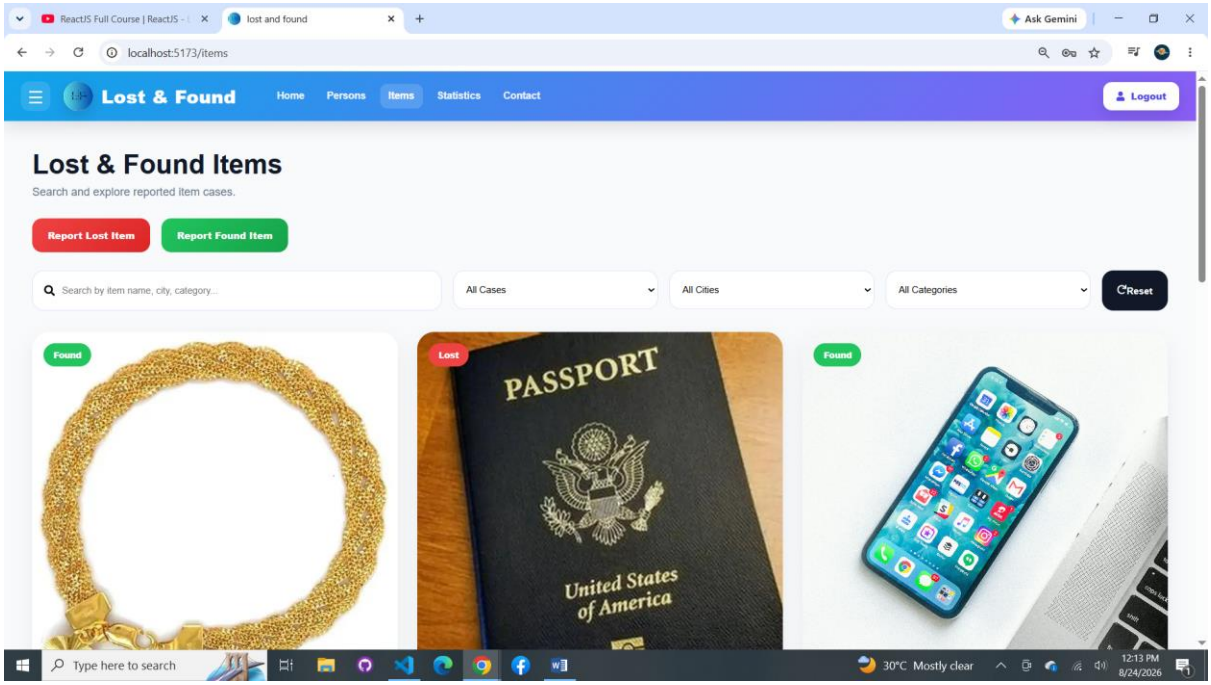
Home Page



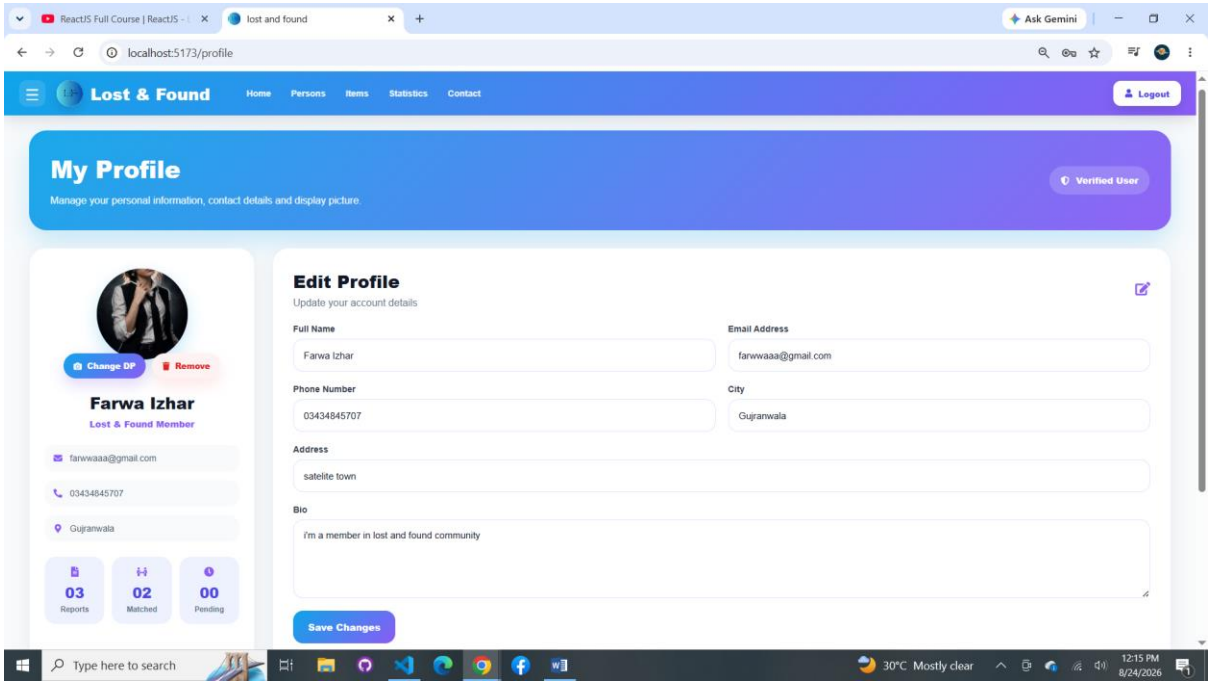
Persons



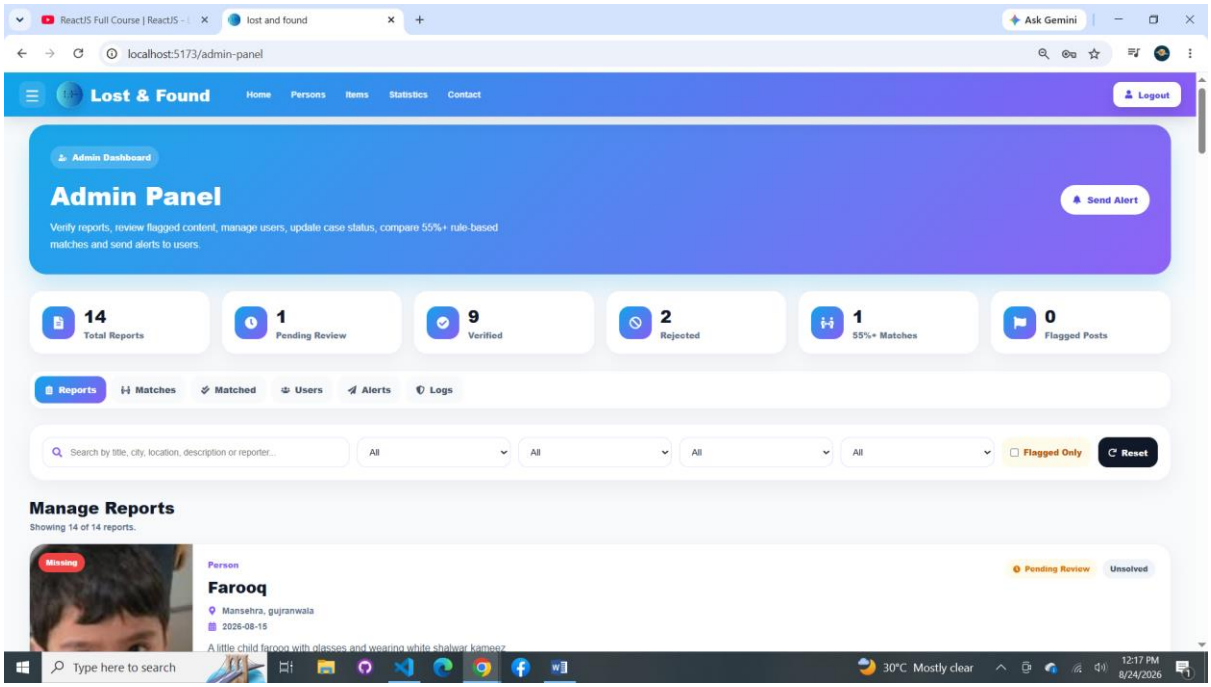
Items



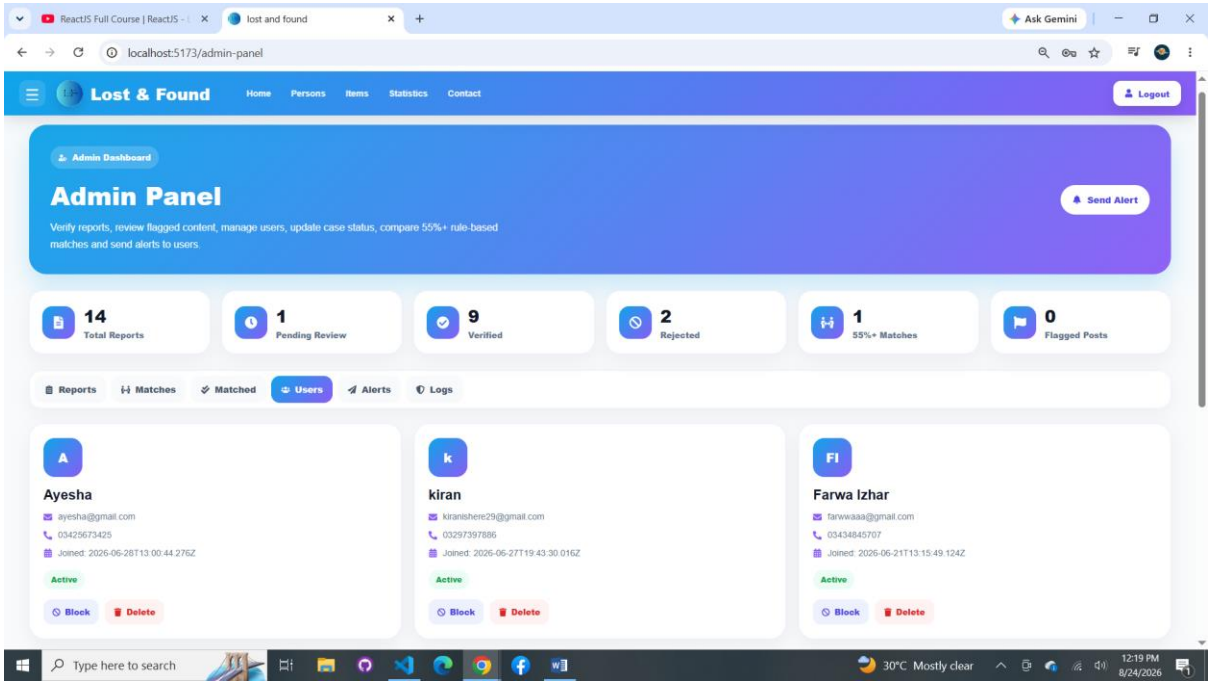
My Profile



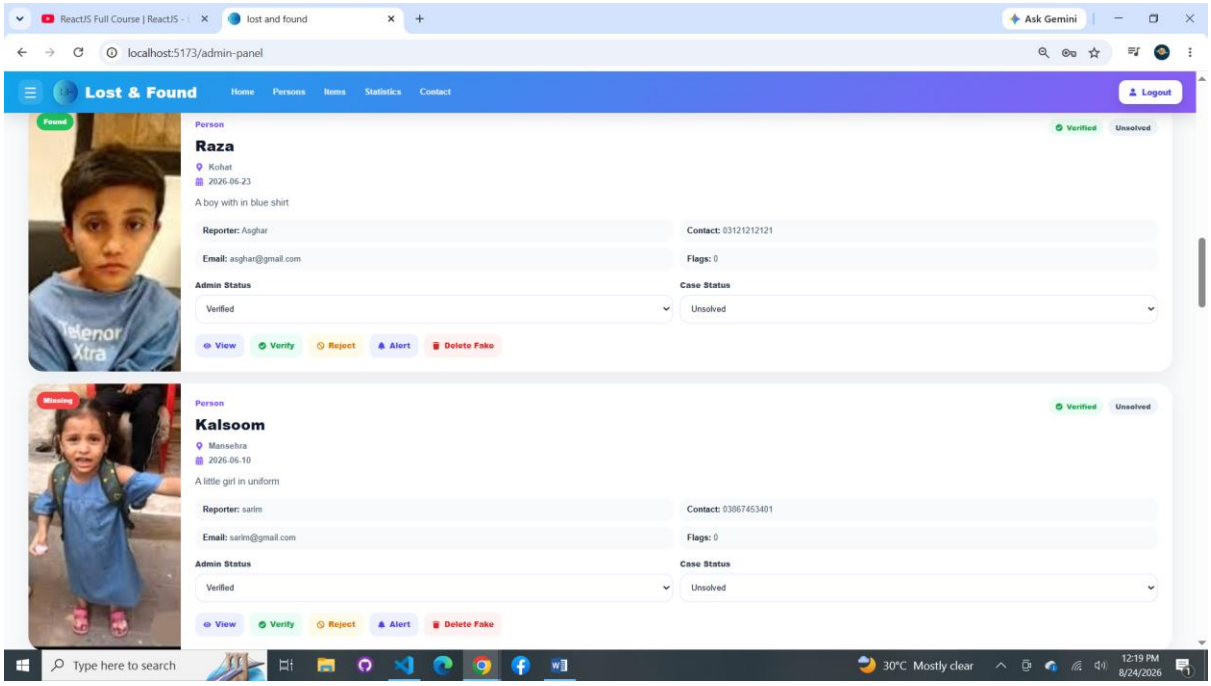
Admin Dashboard



Manage Users



Verify Cases



5.ER Diagram

The ER diagram shows how all the important data objects are connected with each other. It explains what information we store and different parts of the system related.

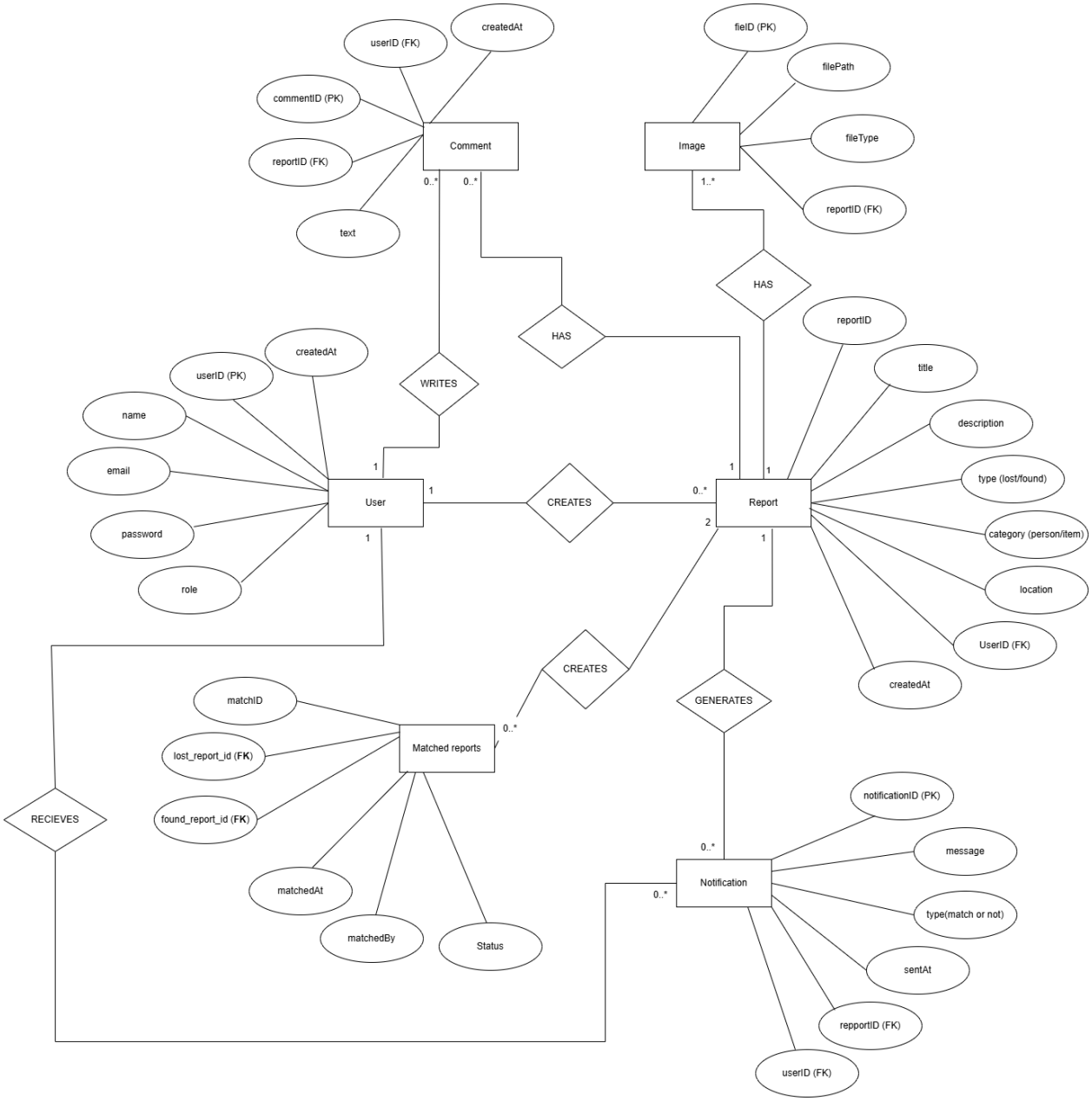


Figure 6: ER Diagram

3.6 Behavioral Model

3.6.1 Sequence Diagram

Description: The Sequence diagram shows how different users and system components interact to complete a process in our system. It displays the order of messages exchanged between the User, Admin and the System when reporting or searching for lost and found persons or items. This Diagram helps to understand step by step how requests are processed and how the system responds. It represents the flow of actions over time to ensure efficiency and manage of lost and found items or persons.

Admin Management Module

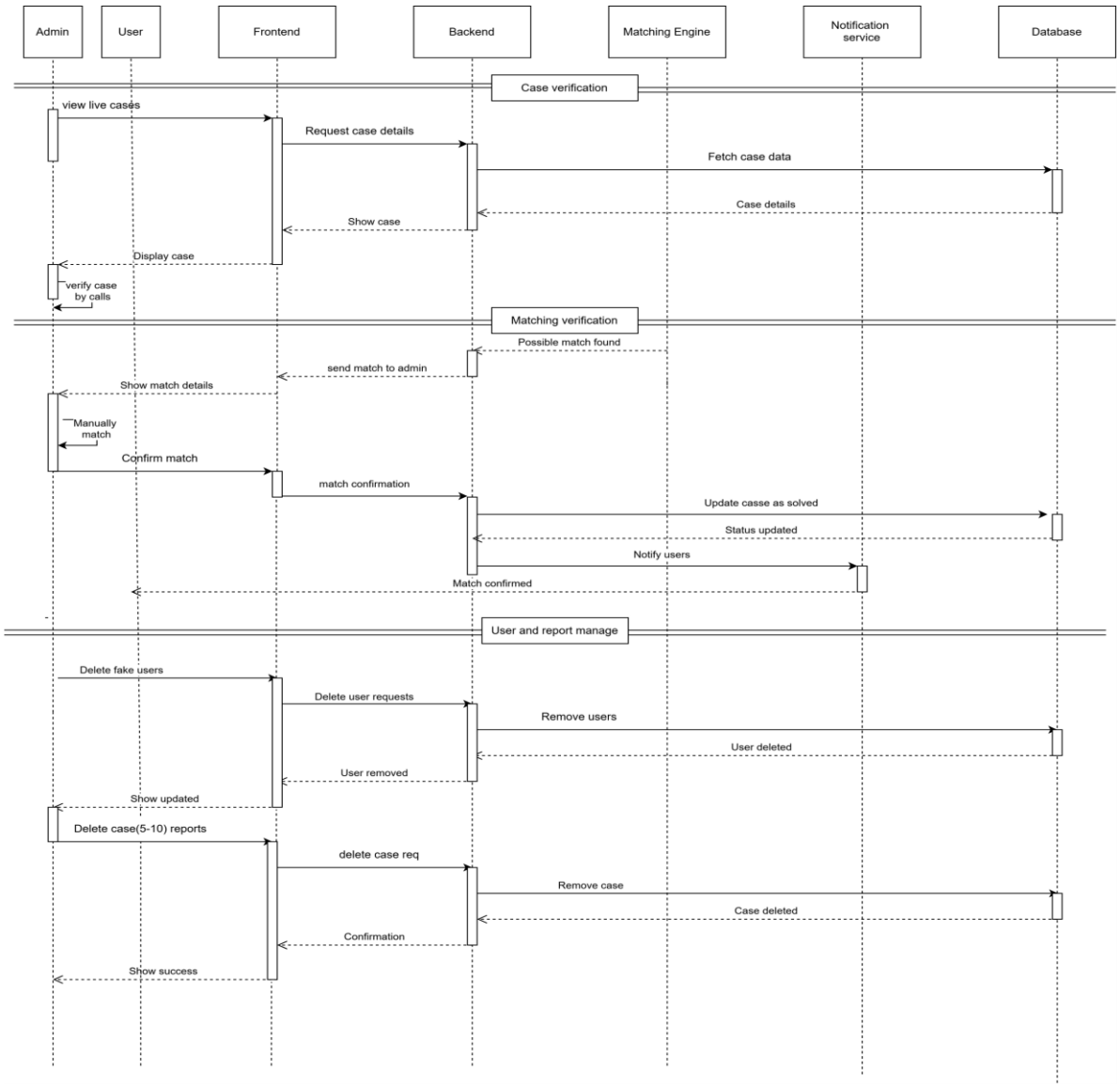


Figure 7: Sequence Diagram 1

User & Authentication Module

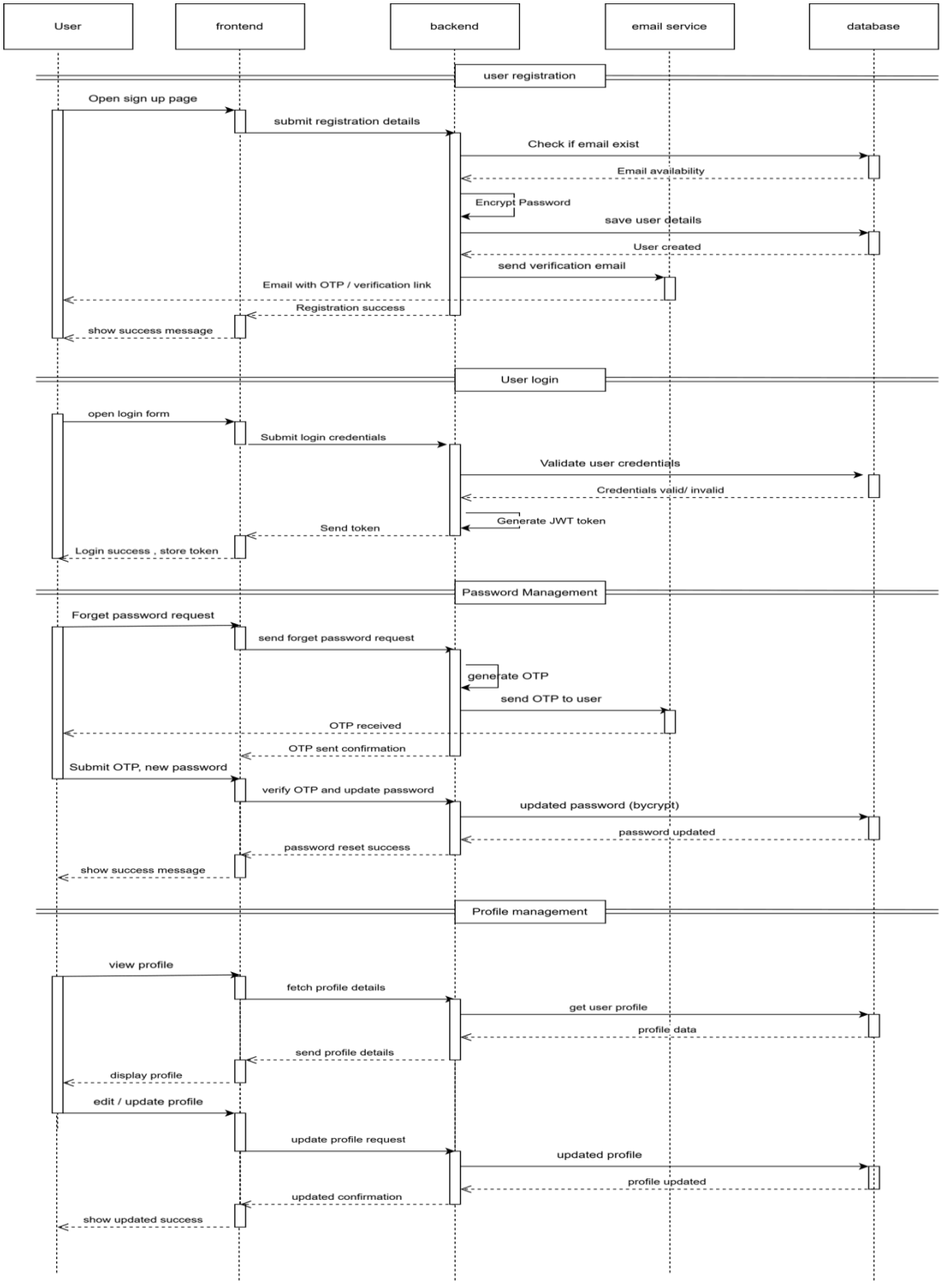


Figure 8: Sequence Diagram 2

Report Management Module

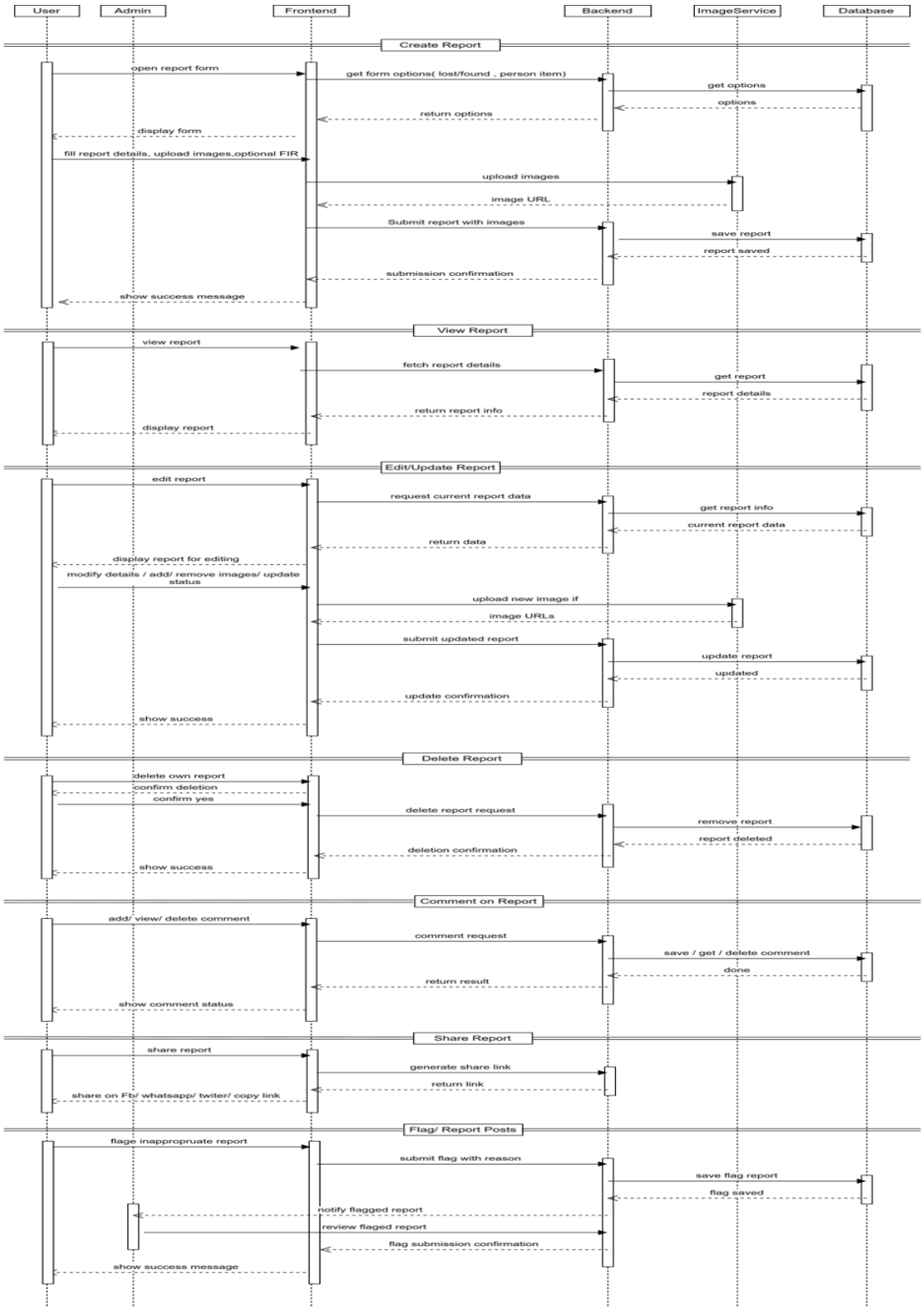


Figure 9: Sequence Diagram 3

Search & Filter Module

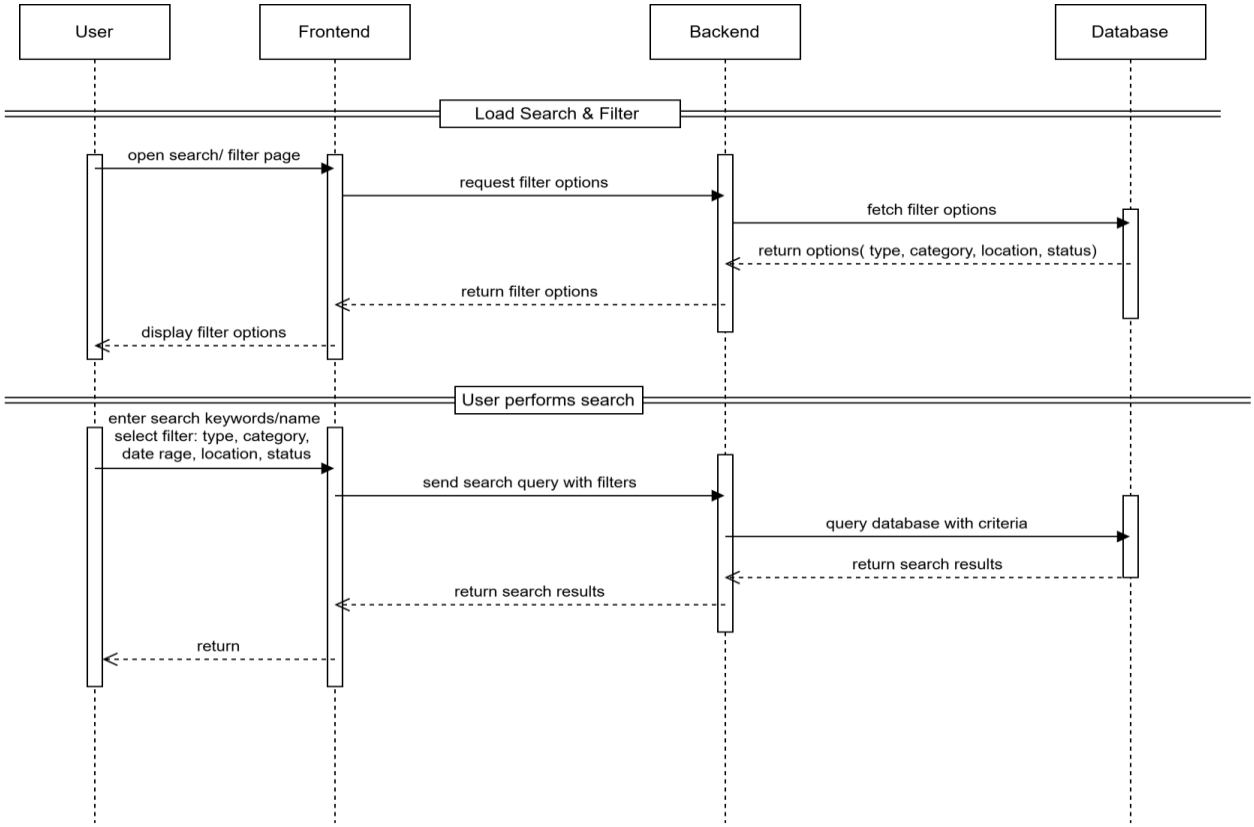


Figure 10: Sequence Diagram 4

Statistics Module

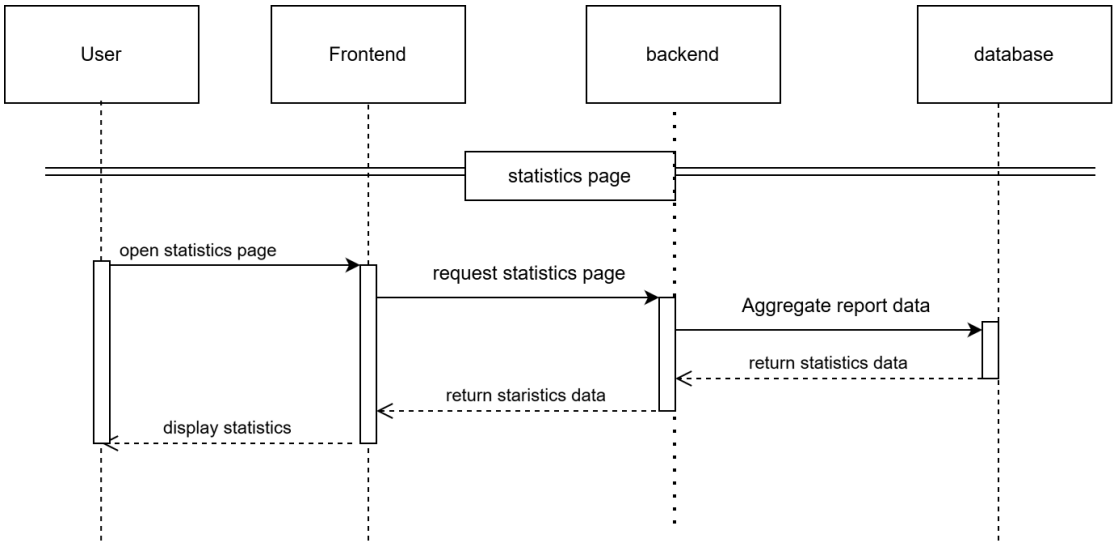


Figure 11: Sequence Diagram 5

3.6.2 State Transition Diagram

Description: A State Transition Diagram shows how the system moves from one state to another based on user actions or events. It explains the flow of states.

In our Project, A **Registered User** logins and start the process. They fill the form and submit the form. The form gets saved and the admin checks it. If the information is correct, it becomes verified; if not, the admin marked it is not verified. The User can update or delete the form.

Unregistered User opens the system without logging it. The User can search, view and share report, and view statistics. The Unregistered User cannot update or delete the form unless they register or log in.

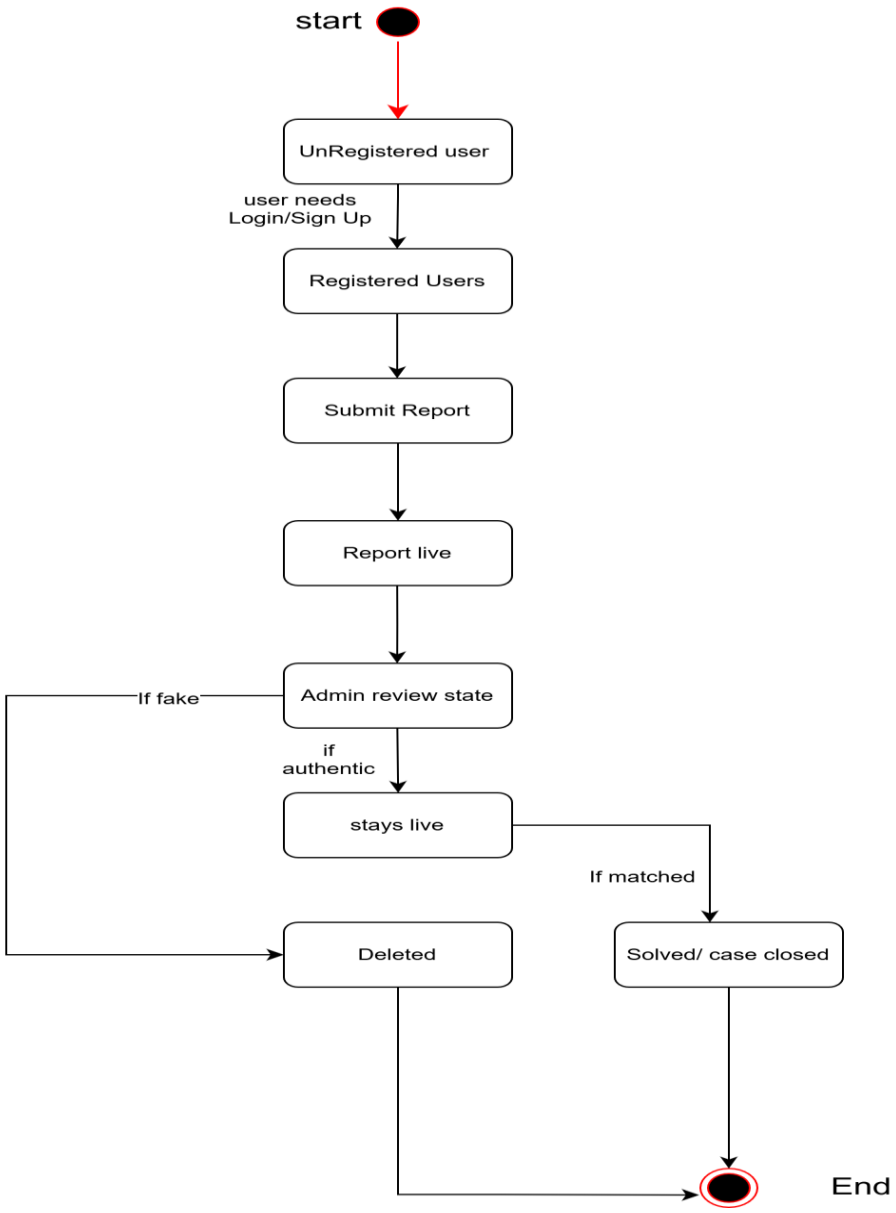


Figure 12: State Transition Diagram

3.7 Design Decisions

Design decisions are the important choices we make while building the system. These choices affect how the system will look, work and perform. The main design decisions according to our lost and found project are:

MERN Stack

We choose MERN Stack for our project because MongoDB stores data easily in a flexible JSON like format, React.js a frontend library that creates a user friendly, responsive interface, Node.js a runtime environment that runs the background code, Express.js a backend framework that manages server-side logic.

Database Design

MongoDB is a NOSQL database that we use to store all the data of our project. The database is split into collections and connecting using ids. This makes updates easy and keeps data correct.

MVC

It is a design pattern that we used to organize our system into three parts:

- **Model:** Represents the data and database structure.
- **View:** Represents the user interface.
- **Controller:** Controller for system logic.

3.8 Summary

The Lost and Found is a web-based application that allows users to report, search and track lost or found items/persons. It uses MongoDB for flexible data storage, MERN Stack for full JavaScript development and follows the MVC architecture for organized and maintaining code. Storing images via file path and indexing important fields for fast searches and performance.

Chapter 4

Implementation

4. Implementation

This chapter describes the implementation of the Lost and Found. The system is developed using MERN Stack which includes MongoDB, Express.js, React.js, and Node.js. The purpose of this implementation is to build a complete web-based application where users can report lost and found persons/items easily.

The system is designed in such a way that it provides authentication, reporting, searching, statistics, matching and admin control functionalities. All components are connected properly to ensure smooth working of the application.

This chapter also explains the main algorithms, external APIs used, and code management of the system.

4.1 Algorithm

Table 35: Algorithm 1

Name	**Algorithm 1: User Registration Component**
Input	User details (name, email, password, confirm password)
Output	User account created or error message
Steps	1. Initialize registration form. 2. User enters details. 3. Validate all fields. 4. Check password match. 5. Send data to server. 6. Check if email exists. 7. If not, store user in database. 8. Return success message. 9. Else, show error.
Algorithm	function Register(name, email, password, confirmPassword) 1. if name OR email OR password OR confirmPassword is empty then 2. return “Validation Error” 3. if password != confirmPassword then 4. return “Passwords do not match” 5. user ← DB.findUserByEmail(email) 6. if user exists then 7. return “Email already registered” 8. hashedPassword ← bcrypt.hash(password) 9. newUser ← {name, email, hashedPassword} 10. DB.save(newUser) 11. return “User registered successfully”

Table 36: Algorithm 2

Name	Algorithm 2: User Login Component
Input	Email and password
Output	Access granted or denied
Steps	1. User enters credentials. 2. Validate inputs. 3. Send request to backend. 4. Verify credentials. 5. Generate token. 6. Redirect to dashboard. 7. Else show error.
Algorithm	function Login(email, password) 1. if email OR password is empty then 2. return “Validation Error” 3. user ← DB.findUserByEmail(email) 4. if user == NULL then 5. return “User not found” 6. if bcrypt.compare(password, user.password) == false then 7. return “Invalid credentials” 8. token ← JWT.generate(user.id) 9. return token

Table 37: Algorithm 3

Name	Algorithm 3: Person Report (Lost/Found Person Component)
Input	Person details (type: lost/found, name, reporter ID card pic, age, gender, description, location, date, image)
Output	Person report successfully created

Steps	1. User navigates to Person Report page. 2. Select report type (Lost or Found). 3. Enter person details (name, age, gender, etc.). 4. Upload image if available. 5. Validate all required fields. 6. Send data to backend API. 7. Store report in database under "Persons" collection. 8. Set report status as "pending". 9. Display confirmation message.

Algorithm	function CreatePersonReport(data)
	
	1. if required fields missing then
	2. return "Error"
	3. if image exists then
	4. imageURL ← Cloudinary.upload(image)
	5. report ← {
	type, name, age, gender,
	description, location, date,
	imageURL, status = "pending"
	}
	6. DB.save(report in Persons collection)
	7. return "Person report created successfully"

Table 38: Algorithm 4

Name	Algorithm 4: Item Report (Lost/Found Item Component)
Input	Item details (type: lost/found, title, reporter ID card pic, category, description, location, date, image)
Output	Item report successfully created
Steps	1. User navigates to Item Report page. 2. Select report type (Lost or Found). 3. Enter item details (title, category, etc.). 4. Upload image if available. 5. Validate input fields. 6. Send data to backend API. 7. Store report in database under "Items" collection. 8. Set report status as "pending". 9. Show success message.

Algorithm	function CreateItemReport(data)
	
	1. if required fields missing then
	2. return “Error”
	3. if image exists then
	4. imageURL ← Cloudinary.upload(image)
	5. report ← {
	type, title, category,
	description, location, date,
	imageURL, status = “pending”
	}
	6. DB.save(report in Items collection)
	7. return “Item report created successfully”

Table 39: Algorithm 5

Name	**Algorithm 5: Rule-Based Matching**
Input	Lost reports and Found reports data
Output	Potential matches
Steps	1. Fetch lost and found reports. 2. Compare category (item/person). 3. Match keywords (title/description etc.). 4. Compare location proximity. 5. Compare date range. 6. Assign matching score. 7. If score > threshold → mark as match. 8. Send to admin for verification.
Algorithm	function MatchReports(lostReports, foundReports) 1. matches ← [] 2. for each lost in lostReports do 3. for each found in foundReports do 4. score ← 0 5. if lost.category == found.category then 6. score ← score + 1 7. if NameMatch(lost.name, found.name) then 8. score ← score + 1 9. if locationMatch(lost.location, found.location) then 10. score ← score + 1 11. if dateClose(lost.date, found.date) then 12. score ← score + 1 13. if score >= threshold then 14. matches.add({lost, found, score}) 15. return matches

Table 40: Algorithm 6

Name	Algorithm 6: Search Reports
Input	Keyword, filters (type, category, location)
Output	Matching reports list
Steps	1. User enters search query. 2. Apply filters. 3. Send request to backend. 4. Query database. 5. Match records. 6. Return results. 7. Display to user.
Algorithm	function SearchReports(keyword, filters) 1. query ← {} 2. if keyword exists then 3. query.title OR description ← keyword 4. apply filters (type, category, location) 5. results ← DB.find(query) 6. if results is empty then 7. return “No records found” 8. return results) 9. return token

Table 41: Algorithm 7

Name	Algorithm 7: Admin Verification
Input	Matched reports
Output	Approved or rejected match
Steps	1. Admin views matched cases. 2. Review details of both reports. 3. Verify similarity manually. 4. Approve or reject match. 5. Update status in database. 6. Notify users.

Algorithm	function VerifyMatch(matchId, decision) 1. if decision == “approve” then 2. match.status ← “approved” 3. update related reports status ← "resolved" 4. sendNotification(users) 5. else 6. match.status ← “rejected” 7. DB.update(match) 8. return “Verification completed”

Table 42: Algorithm 8

Name	**Algorithm 8: Reset Password**
Input	User email
Output	Password reset successfully
Steps	1. User enters email. 2. Validate email. 3. Generate reset token. 4. Send reset link/email. 5. User enters new password. 6. Validate password. 7. Update in database. 8. Show success message.
Algorithm	function ResetPassword(email) 1. if email is empty then 2. return “Validation Error” 3. user ← DB.findUserByEmail(email) 4. if user == NULL then 5. return “User not found” 6. token ← generateResetToken() 7. sendEmail(email, token) 8. user enters newPassword 9. hashedPassword ← bcrypt.hash(newPassword) 10. update user password in DB 11. return “Password reset successful”

Table 43: Algorithm 9

Name	Algorithm 9: User Profile Management
Input	Updated user details
Output	Profile updated
Steps	1. User opens profile. 2. Edit details. 3. Validate inputs. 4. Send update request. 5. Save changes in database. 6. Show confirmation.
Algorithm	function UpdateProfile(userId, updatedData) 1. user ← DB.find(userId) 2. if user == NULL then 3. return “User not found” 4. validate(updatedData) 5. update user fields with updatedData 6. DB.save(user) 7. return “Profile updated successfully”

Table 44: Algorithm 10

Name	Algorithm 10: Statistics / Analysis Generation
Input	Reports data (persons and items)
Output	Analytical statistics (counts, trends, summaries)
Steps	1. Fetch all reports from database. 2. Separate data into categories (persons/items, lost/found). 3. Count total reports for each category. 4. Calculate missing, found, resolved cases. 5. Analyze trends based on location and date. 6. Generate summary statistics. 7. Send processed data to frontend. 8. Display results using charts/graphs.

Algorithm	Function Statistics() 1. reports ← DB.getAllReports() 2. persons ← filter reports where type = “person” 3. items ← filter reports where type = “item” 4. lostCount ← count where status = “lost” 5. foundCount ← count where status = “found” 6. resolvedCount ← count where status = “resolved” 7. analyze trends by location and date 8. stats ← {persons, items, lostCount, foundCount, resolvedCount} 9. return stats

Table 45: Algorithm 11

Name	**Algorithm 11: User Logout**
Input	User session/token
Output	User logged out
Steps	1. User clicks logout. 2. Remove token. 3. Clear session. 4. Redirect to login page.
Algorithm	function Logout(userToken) 1. if userToken exists then 2. remove token from storage 3. clear user session 4. redirect to login page 5. return “User logged out successfully”

4.2 External APIs

Table 46: External APIs

Name of API	Description of API	Purpose of usage	List down the API endpoints/ function class in which it is used
Nodemailer (Node.js module)	Nodemailer is a widely used Node.js library that allows developers to send emails directly from the server using SMTP (Simple Mail Transfer Protocol). It supports secure email transmission through authentication, encryption (TLS/SSL), and integration with various email services like Gmail. It also allows sending dynamic content such as alerts, verification links, and notifications programmatically.	Used to send system generated emails such as password reset, notifications and report matching alerts.	/api/notify, sendEmail(), mailService.js

4.3 Code Repository

Git Repository Link: https://github.com/Farwa07/Lost-and-Found

It's is a public repository; entire code is there and will be updated on daily basis.

4.4 Summary

This chapter provided a comprehensive overview of the implementation of the Lost and Found System. The system is developed using the MERN stack and incorporates essential functionalities such as user authentication, item reporting, and search operations. Additionally, the algorithms, external APIs, and code repository structure were discussed to provide a clear understanding of the system's development process. The implementation ensures a scalable, secure, and user-friendly application.

Chapter 5

Testing and Evaluation

Testing and Evaluation

This chapter presents the testing and evaluation process. It ensures that our platform functions are correct and reliable. Software testing is ensured through verification and validation (V&V). The V&V phase confirms that the system meets all specified requirements. The test cases are designed to ensure that the system is stable, user-friendly, and free from critical errors.

To maintain high quality, all test cases follow specific best practices:

- **Independent Tests:** Each test case is designed to run independently. This ensures no interference between different test processes.
- **Meaningful Assertions:** Clear and specific assertions are used to determine if a test passes or fails.
- **Repeatability:** Tests are designed to be repeatable. This provides consistent results across multiple executions.
- **Data Management:** Setup and teardown methods are implemented to manage test data effectively.

The testing process is categorized into several levels:

1. **Unit Testing (UT):** It focuses on testing individual software components. This testing validates that each unit performs according to its design.
2. **Functional Testing (FT):** It ensures that every module meets the functional requirements and stakeholders' expectations.
3. **Integration Testing (IT):** It assesses the interaction between classes and functions. This testing also ensures that multiple components collectively produce the desired results.
4. **Performance Testing (PT):** It measures system reliability under stress. It also confirms that the system performs efficiently under expected load.

All test cases for these categories are documented using a standardized template. This systematic approach ensures complete traceability and consistency throughout the evaluation phase.

5.1 Unit Testing (UT)

Unit testing is the fundamental level of software testing. In UT, individual units or components of the application are tested in isolation. A "unit" in our project represents the smallest testable part of the software, such as a specific function or a class method.

The primary purpose is to validate that each specific unit performs exactly as designed, it also validates that each unit meets its functional specifications before it is integrated into the larger system.

5.1.1 All Unit Test Cases (UT) Brief Specifications

This table shows the brief description for the Units and their inputs and expected results

Table 47: All Unit Test Cases

Test ID	Module/Function	Input	Expected Result
UT-01	User Registration	Valid email, strong password	User stored in MongoDB, status 201
UT-02	User Registration	Existing email	Error: "Email already exists"
UT-03	User Login	Correct email/password	JWT token generated, session active
UT-04	User Login	Wrong password	Error: "Invalid credentials"
UT-05	Create Report	Full report data + Image	Report saved, image path returned
UT-06	Add Comment	Valid text, Report ID	Comment saved and linked to Report ID
UT-07	Search Module	Valid search keyword	Returns list of matching reports
UT-08	Reset Password	Valid OTP	Password updated successfully
UT-09	Admin Dashboard	Delete Report request	Report removed from MongoDB
UT-10	Notification	Trigger Match Event	Notification created for relevant user

5.1.2 Test Case 1

Field	UT-01: User Registration
Testcase ID	UT-01
Requirement ID	RQ-003
Title	User Registration
Description	Validation of new user account creation logic.
Objective	Ensure valid user data is stored correctly in MongoDB.
Driver/precondition	Database server is running and connected.

Test steps	1. Navigate to Registration page. 2. Enter valid user details. 3. Click Submit.
Input	Valid Name, Email, Phone, Password.
Expected Results	Status 201, User saved in database.
Actual Result	Pass
Test Status (Fail/Pass)	Pass

5.1.3 Test Case 2

Field	**UT-02: Duplicate Registration**
Testcase ID	UT-02
Requirement ID	RQ-003
Title	Duplicate Registration
Description	Validation of email uniqueness check during signup.
Objective	Ensure no duplicate accounts are created.
Driver/precondition	User already exists in database.
Test steps	1. Navigate to Registration. 2. Enter existing email. 3. Click Submit.
Input	Already registered email.
Expected Results	Error: "Email already exists".
Actual Result	Pass
Test Status (Fail/Pass)	Pass

5.1.4 Test Case 3

Field	**UT-03: User Login**
Testcase ID	UT-03
Requirement ID	RQ-003

Title	User Login
Description	Validation of credentials and session creation.
Objective	Ensure secure token generation .
Driver/precondition	User is registered.
Test steps	1. Navigate to Login. 2. Enter correct email/password. 3. Click Login.
Input	Correct credentials.
Expected Results	JWT token generated, session active.
Actual Result	Pass
Test Status (Fail/Pass)	Pass

5.1.5 Test Case 4

Field	**UT-04: Invalid Login**
Testcase ID	UT-04
Requirement ID	RQ-003
Title	Invalid Login
Description	Validation of login rejection for wrong passwords.
Objective	Reject unauthorized access.
Driver/precondition	Registered account exists.
Test steps	1. Enter email. 2. Enter wrong password. 3. Click Login.
Input	Incorrect password.
Expected Results	Error: "Invalid credentials".
Actual Result	Pass
Test Status (Fail/Pass)	Pass

5.1.6 Test Case 5

Field	UT-05: Create Report
Testcase ID	UT-05
Requirement ID	RQ-001
Title	Create Report
Description	Validation of report data and image upload.
Objective	Ensure report saved with file path .
Driver/precondition	User is logged in.
Test steps	1. Open 'Add Report' form. 2. Fill details. 3. Attach image. 4. Submit.
Input	Valid form data and Image file.
Expected Results	Report saved, image path returned.
Actual Result	Pass
Test Status (Fail/Pass)	Pass

5.1.7 Test Case 6

Field	UT-06: Add Comment
Testcase ID	UT-06
Requirement ID	RQ-004
Title	Add Comment
Description	Validation of comment saving to DB.
Objective	Link comment to specific Report ID .
Driver/precondition	Report post is visible.
Test steps	1. View report. 2. Type comment. 3. Submit.
Input	Valid comment text.
Expected Results	Comment saved and linked to Report ID.
Actual Result	Pass
Test Status (Fail/Pass)	Pass

5.1.8 Test Case 7

Field	UT-07: Search Module
Testcase ID	UT-07
Requirement ID	RQ-002
Title	Search Module
Description	Querying reports via search keywords.
Objective	Retrieve accurate search results .
Driver/precondition	Reports exist in database.
Test steps	1. Open search page. 2. Enter keyword. 3. Search.
Input	Search keyword.
Expected Results	Returns list of matching reports.

Actual Result	Pass
Test Status (Fail/Pass)	Pass

5.1.9 Test Case 8

Field	**UT-08: Reset Password**
Testcase ID	UT-08
Requirement ID	RQ-004
Title	Reset Password
Description	OTP-based password recovery.
Objective	Securely update password.
Driver/precondition	OTP received via email.
Test steps	1. Enter OTP. 2. Enter new password. 3. Submit.
Input	Correct OTP and new password.
Expected Results	Password updated successfully.
Actual Result	Pass
Test Status (Fail/Pass)	Pass

5.1.10 Test Case 9

Field	UT-09: Admin Dashboard
Testcase ID	UT-09
Requirement ID	RQ-005
Title	Delete Report
Description	Admin-side report removal.
Objective	Remove irrelevant reports from MongoDB.
Driver/precondition	Admin is logged in.
Test steps	1. View report. 2. Click Delete. 3. Confirm.
Input	Delete request.
Expected Results	Report removed from database.
Actual Result	Pass
Test Status (Fail/Pass)	Pass

5.1.11 Test Case 10

Field	UT-10: Notification
Testcase ID	UT-10
Requirement ID	RQ-005
Title	Notification
Description	Trigger notification on match event.
Objective	Alert users on match .
Driver/precondition	Match condition met.
Test steps	1. Trigger match. 2. Check notifications.
Input	Match event.
Expected Results	Notification created for user.
Actual Result	Pass
Test Status (Fail/Pass)	Pass

5.2 Functional Testing (FT)

Functional testing is the phase where we evaluate the system's behavior against the specified functional requirements. The Unit Testing focuses on individual code blocks, while Functional Testing validates the entire application from the stakeholder's perspective. The goal is to confirm that the system correctly implements the features defined in the requirements specification.

Functional Testing Approach

We test the system by providing inputs and observing the outputs based on actual business workflows. Each feature is tested to ensure that the interactions between the user and the system result in the expected outcome.

5.2.1 Functional Test Cases (FT) Brief Description

The brief description of all functional test cases given below:

Table 48: All functional Test Cases

Test ID	Feature / Module	Test Scenario	Expected Result
FT-01	Report Submission	User submits a missing person report with all details and images.	System stores the report, triggers the admin verification process, and displays it publicly.
FT-02	Search & Filtering	User searches for an item by location and category.	System retrieves and displays only the reports matching the specified criteria.
FT-03	Admin Verification	Admin reviews and approves a reported case.	The report status changes from "Pending" to "Active" and becomes visible to all users.
FT-04	Case Matching	Rule based matches a 'Lost' report with a 'Found' report.	Admin updates both statuses to "Matched" and sends email alerts to both reporters.
FT-05	User Interaction	User adds a comment to a specific report post. Admin	The comment is successfully saved and immediately visible to other users viewing the same report.
FT-06	Statistics View	User views the monthly statistics page.	System displays accurate visual charts showing missing, found, and solved cases for the month.

5.2.2 Test Case 1

Field	FT-01: Report Submission
Testcase ID	FT-01
Requirement ID	RQ-001
Title	Report Submission
Description	User submits a missing/found report with details.
Objective	Verify that report data and images are stored successfully .
Driver/precondition	User is registered and logged in.
Test steps	1. Navigate to 'Add Report'. 2. Fill form details. 3. Upload image. 4. Submit.
Input	Valid text data, valid image file.
Expected Results	Report saved in MongoDB, image path stored, status set to 'Pending'.
Actual Result	Pass
Test Status (Fail/Pass)	Pass

5.2.3 Test Case 2

Field	**FT-02: Search & Filtering**
Testcase ID	FT-02
Requirement ID	RQ-002
Title	Search & Filtering
Description	Filter reports based on category or location.
Objective	Verify accurate data retrieval based on filters .
Driver/precondition	Reports exist in the system.
Test steps	1. Open search page. 2. apply filters. 3. Click filter button.
Input	Search parameters (Category/Location/name/).
Expected Results	Displays only reports matching the selected filters.
Actual Result	Pass
Test Status (Fail/Pass)	Pass

5.2.4 Test Case 3

Field	**FT-03: Admin Verification**
Testcase ID	FT-03
Requirement ID	RQ-005
Title	Admin Verification
Description	Admin reviews and verifies user reports.
Objective	Ensure fake reports are filtered out .
Driver/precondition	Admin is logged in.
Test steps	1. Open Admin Dashboard. 2. Select pending report.

	3. Click 'Verify'.
Input	Verification command.
Expected Results	Report status updated to 'Verified/Active'.
Actual Result	Pass
Test Status (Fail/Pass)	Pass

5.2.5 Test Case 4

Field	**FT-04: Case Matching**
Testcase ID	FT-04
Requirement ID	RQ-005
Title	Case Matching
Description	system matches 'Lost' report with 'Found' report.
Objective	Ensure rule based correct matching.
Driver/precondition	Two matching reports exist.
Test steps	1. Select two reports. 2. Click 'Match'.
Input	Match execution command.
Expected Results	Both statuses updated, notification sent to users.
Actual Result	pass
Test Status (Fail/Pass)	pass

5.2.6 Test Case 5

Field	FT-05: User Interaction
Testcase ID	FT-05
Requirement ID	RQ-004
Title	User Interaction (Comments)(share)(flag content)
Description	User posts a comment on a report.
Objective	Verify real-time comment visibility.
Driver/precondition	User is logged in.
Test steps	1. View report. 2. Enter comment. 3. Post.
Input	Comment text.
Expected Results	Comment saved and visible on the report page.
Actual Result	Pass
Test Status (Fail/Pass)	Pass

5.2.7 Test Case 6

Field	FT-06: Statistics Display
Testcase ID	FT-06
Requirement ID	RQ-002
Title	Statistics View
Description	Visual representation of missing/found cases.
Objective	Ensure statistical charts render correct data.
Driver/precondition	Data exists in the system.
Test steps	1. Navigate to Dashboard/Statistics.
Input	System data.
Expected Results	Visual charts (graphs) accurately represent monthly data.

Actual Result	Pass
Test Status (Fail/Pass)	Pass

5.3 Integration Test Cases (IT)

Integration testing is a crucial phase where individual software modules are combined and tested as a group. The primary goal is to expose defects in the interfaces and interaction between integrated components. The main purpose is to verify that different modules work together seamlessly as intended in the system architecture.

It specifically focuses on checking the data flow between the different components like frontend, backend, and database.

The details of each integration test case are given below:

5.3.1 Test Case

Field	**IT-01: Auth & User Profile**
Testcase ID	IT-01
Requirement ID	TR-003, TR-004
Title	Authentication & Profile Integration
Description	Validates JWT token persistence and user profile data fetching.
Objective	Ensure the authenticated user can access their specific profile data.
Driver/precondition	User is logged in with a valid JWT token.
Test steps	1. Log in. 2. Navigate to 'My Profile'. 3. Observe API call to backend.
Input	Valid Authorization Header (JWT).
Expected Results	Backend returns user profile data; UI displays user details correctly.
Actual Result	Pass
Test Status (Fail/Pass)	Pass

5.3.2 Test Case 2

Field	IT-02: Reporting & Image Service
Testcase ID	IT-02
Requirement ID	TR-001
Title	Report Submission & Image Upload
Description	Integrates form data submission with cloudinary image upload service.
Objective	Ensure image URL is correctly linked to the report in the database.
Driver/precondition	Frontend form is ready; cloudinary storage is configured.
Test steps	1. Fill report form. 2. Select image. 3. Submit.
Input	Form text data + Image file.
Expected Results	Image uploaded to server; database entry contains correct file path.
Actual Result	Pass
Test Status (Fail/Pass)	Pass

5.3.3 Test Case 3

Field	IT-03: Search & Database
Testcase ID	IT-03
Requirement ID	TR-002
Title	Search/Filter & MongoDB Integration
Description	Validates integration between search UI and database query logic.
Objective	Ensure database returns filtered data correctly to the search UI.
Driver/precondition	Reports exist in MongoDB.
Test steps	1. Apply filter (e.g., 'Lost'). 2. Execute search.
Input	Filter query parameters.
Expected Results	Backend executes correct Mongoose query; frontend renders filtered reports.
Actual Result	Pass
Test Status (Fail/Pass)	Pass

5.3.4 Test Case 4

Field	IT-04: Matching & Notifications
Testcase ID	IT-04
Requirement ID	TR-005
Title	Matching Engine & Notification Service
Description	Integrates Admin 'Match' action with Nodemailer email service.
Objective	Ensure match confirmation triggers immediate email alerts.
Driver/precondition	SMTP server configuration is active.
Test steps	1. Admin confirms match. 2. Check notification service status.
Input	Admin Match Action.
Expected Results	User receives match alert email; notification flag set to 'True'.
Actual Result	Pass
Test Status (Fail/Pass)	Pass

5.3.5 Test Case 5

Field	IT-05: Commenting & Report Linkage
Testcase ID	IT-05
Requirement ID	TR-004
Title	Commenting & Report Linkage
Description	Integrates comment functionality with the specific Report document.
Objective	Ensure comments appear only on the relevant report post.
Driver/precondition	A report post exists.
Test steps	1. Open specific report. 2. Add comment. 3. Reload page.
Input	Comment text + Report ID.
Expected Results	Comment is saved to DB linked to that specific Report ID.
Actual Result	Pass
Test Status (Fail/Pass)	Pass

5.4 Performance Testing

Performance testing is conducted to determine how a system performs in specific load times. It evaluates the system's responsiveness and stability under a particular workload. This testing phase ensures the tested application is fast and reliable. It also measures the capability of the system to handle expected user traffic without degrading in quality.

The main purpose of this testing is to identify system bottlenecks like slow response times, memory leaks, or stability issues, before the system is fully deployed.

Common Types:

- **Load Testing:** Testing the system under expected user traffic levels to ensure it meets performance requirements.
- **Stress Testing:** Pushing the system beyond its normal operational capacity to find the breaking point and ensure it recovers gracefully.
- **Scalability Testing:** Determining if the system can effectively increase its capacity to handle a growing amount of data or users.
- **Reliability Testing:** Running the application for 24 hours under continuous load. Detect memory leaks or performance degradation over time.

Following are the test cases for these typed of performance testing:

5.4.1 Test Case 1

Field	**PT-01: Load Testing**
Testcase ID	PT-01
Requirement ID	N/A (Performance Req)
Title	Concurrent User Load
Description	Measuring system response time under multiple concurrent users.
Objective	Assess system stability under normal to high traffic.
Driver/precondition	Performance testing tool (e.g., JMeter) is configured.
Test steps	1. Simulate 50 concurrent users. 2. Perform search/browse actions. 3. Monitor server metrics.
Input	Simulated load of 50 users.
Expected Results	Response time < 2 seconds; zero system crashes.
Actual Result	Pass
Test Status (Fail/Pass)	Pass

5.4.1 Test Case 2

Field	PT-02: Stress Testing
Testcase ID	PT-02
Requirement ID	N/A (Performance Req)
Title	Breakpoint Determination
Description	Gradually increasing load until the system performance degrades.
Objective	Identify the maximum number of users the system can handle.
Driver/precondition	Monitoring dashboard is active.
Test steps	1. Start with 50 users. 2. Increment by 10 users. 3. Note the crash point.
Input	Increasing load (50, 60, 70... users).
Expected Results	Identify the threshold where latency > 5 seconds or error rate increases.
Actual Result	Pass
Test Status (Fail/Pass)	Pass

5.4.1 Test Case 3

Field	PT-03: Scalability Testing
Testcase ID	PT-03
Requirement ID	N/A (Performance Req)
Title	Database Retrieval Speed
Description	Measuring time taken to retrieve reports from MongoDB with large datasets.
Objective	Ensure search speed remains optimal as data grows.
Driver/precondition	Database is populated with 1000+ records.
Test steps	1. Populate DB. 2. Execute keyword search.

	3. Record time.
Input	Search keyword query.
Expected Results	Search results returned within 1.5 seconds.
Actual Result	Pass
Test Status (Fail/Pass)	Pass

5.3.4 Test Case 4

Field	**PT-04: Reliability Testing**
Testcase ID	PT-04
Requirement ID	N/A (Performance Req)
Title	Long Duration Stability
Description	Running the application for 24 hours under continuous load.
Objective	Detect memory leaks or performance degradation over time.
Driver/precondition	System is idle and clean.
Test steps	1. Start constant light load. 2. Monitor resource usage for 24 hours.
Input	Continuous small load.
Expected Results	No memory leaks; system remains operational.
Actual Result	Pass
Test Status (Fail/Pass)	Pass

5.5 Summary

This chapter focuses on testing and evaluation of our project, Unit testing, Integration testing, and functional testing of several units and components were conducted with satisfactory results.

Chapter 6

System Conversion

6. Introduction

System conversion is the process of changing from one information system to another. This involves transferring data, processes, and operations from an old system to a new one.

6.1 Conversion Method

For our project the Pilot Conversion method was adopted.

- In our project, we first implement the system on a small group of users before full deployment.
- Since our system is new, there is no proper old centralized system.
- Testing with limited users helps identify bugs, improve UI/UX, check report matching and notification system
- Reduces risk before full deployment.

6.2 Deployment

The steps required to deploy the project on docker are as follows:

1. **Build Frontend**
   The frontend was build using npm run build command.
2. **Prepare backend**
   The backend was set up using Node.js and configure APIs using Express.
3. **Database Setup**
   Setup MongoDB and store data in MongoDB collections.
4. **Environment Configuration**
   Set environment variables JWT Secret, Email credentials and Database URI.
5. **Docker Deployment**
   Create docker file for frontend and backend, build docker images and run containers.
6. **Run Application**
   Start backend server and connect frontend with backend.
7. **Testing**
   Check features:
   - User login/signup
   - Create report
   - Upload image
   - Search/filter
   - Admin features

6.2.1 Data Conversion

The data conversion process involved:

- **Data Extraction:** Collect existing records of missing and found persons or items from social media posts and user enters manually missing or found reports.
- **Data Cleaning:** Remove duplicate entries and ensure all images are in the correct format.
- **Data Loading:** Use MongoDB import tool or backend script to insert cleaned data into your MongoDB collections.
- **Backup & Restore:** Use mongo dump to create a backup of the initial database and mongo restore to ensure the data + be recovered in case of failure.

6.2.2 Training

To train the users, user manual of two main use cases of the system can be presented:

Use Case 1: Report missing and found person or items

- User login/signup.
- Click create report.
- Enter details.
- Upload image.
- Submit report.

Use Case 2: Verification and Matching

- Admin login.
- View submitted reports.
- Verify report details and images.
- Check fake or incomplete reports.
- System generates possible matches.
- Admin compares lost and found reports.
- Confirm the match.
- Update report status.
- Send notification to users.

6.3 Post Deployment Testing

Post deployment testing includes the following steps:

- Verify that the homepage loads and the login/signup functionality works.
- Create a dummy lost and found report and verify it appears in the database and on the public dashboard.
- Ensure chart.js correctly visualizes the statistics.
- Ensures that an email is successfully sent to the user when the admin updates a case status.

6.4 Challenges

Challenges	Solutions
Fake reports	Admin reviews and verifies reports and if a report is fake, it is rejected or deleted
Inappropriate Comments	Admin manages inappropriate comments by monitoring content, using filters and can remove or hide comments.

6.5 Summary

The lost and found system is deployed using the MERN stack with docker support. We selected pilot conversion to reduce risk and test the system with limited users. Some challenges like fake reports, image upload issues and matching accuracy were resolved using admin control, validation and improved logic, ensuring that the system could be used reliably in a real-world or academic setting.

Chapter 7

Conclusion

7. Introduction

This chapter provides a final overview of the project. It evaluates the extent to which the initial project goals and objectives were achieved. It also outlines potential opportunities for future enhancements. It serves as the closing argument for the development of the "Lost and Found" system and confirms that the solution effectively addresses the problem statement. This chapter also highlights the significance of the contributions made.

7.1 Evaluation

In this section, we provide a structured evaluation of each objective defined in Chapter 1 with implemented features of the system. This confirms that the project was executed according to the original plan.

This section includes the full scope of our traceability. We evaluate our success by mapping our BRs (Business Requirements) to the actual implementation status.

Table 49: mapping of BRs with actual implementation

Objective	**Req. ID**	**Status**	**Implementation Detail**
Centralized Reporting	BR001	Completed	Web interface for reporting with FIR field.
Statistical Analysis	BR002	Completed	Chart.js implementation for monthly trends.
Secure Authentication	BR003	Completed	JWT and bcrypt implementation.
User Interaction	BR004	Completed	Commenting and social sharing modules.
Matching & Verification	BR005	Completed	Admin dashboard matching engine and email alerts.

7.2 Traceability Matrix

The traceability matrix ensures that every requirement identified during the analysis phase has been mapped to its corresponding design specification, source code, and test case. This demonstrates that all requirements are fully accounted for in the final product. This table provides the consolidated view requested in the manual.

Table 50: Traceability Matrix

Req. ID	Description	Design Component	Code File	Test ID
BR001	Report Submission	Report Module	ReportController.js	UT-05
BR002	Search/Stats	Search/Stats Module	StatsService.js	UT-07
BR003	Authentication	Auth Module	AuthController.js	UT-03
BR004	User Interaction	Comment/Share Module	CommentService.js	UT-06
BR005	Admin/Matching	Admin Dashboard	ReportMatch.js	UT-10

Analysis of Traceability

Vertical Integration: Every requirement listed in Chapter 2 (Functional Requirements) has been tracked through the design models (Chapter 3) and validated in the testing phase (Chapter 5).

Gap Mitigation: During the development lifecycle, the RTM allowed us to identify gaps. This RTM help out to find requirements that were initially specified. For example, the need for a separate 'Match' entity in our database to handle many-to-many relationships, which was later addressed in the design.

Verification and Validation: By mapping the Test Cases back to Business Requirements, the Verification and Validation have done. For example, the UT-10 for the Notification service mapped back to BR005. This provides evidence that the admin’s ability to send alerts is not just a conceptual feature. It is a verified and functional system component now.

7.3 Conclusion

The "Lost and Found" system has successfully achieved its primary goal of providing a centralized, secure, and user-friendly platform. It reunites people with their missing items or loved ones. The MERN stack effectively mitigates the issues of fragmented data and slow verification processes found in other solutions. The project contributes significantly to community-based recovery efforts. It completely fulfills the functional and non-functional requirements established at the start.

The fulfilment of business requirements has successfully transformed from a conceptual proposal into a fully functional MERN-stack application. The project has achieved its goal by

providing a free-of-cost and centralized platform for missing persons and items. It has filled the gaps found in existing platforms and social media alternatives.

7.4 Future Work

While the system is functional, there are several opportunities for future enhancements to improve scalability and utility:

- **Live GPS Integration:** Implementing real-time location tracking for found items or persons to improve accuracy.
- **Advanced AI Matching:** Utilizing machine learning algorithms to automatically identify similar images and items, reducing manual administrative load.
- **Mobile Application:** Expanding the platform into a native mobile application (Android/iOS) for better accessibility.
- **Government Integration:** Establishing secure APIs to connect with police or NGO databases for verified and faster recovery processes.
- **AI-Based Verification:** Enhancing the matching engine with image recognition AI to improve the accuracy of item identification.
- **Notification Scaling:** Implementing Push Notifications for faster mobile-ready alerts beyond email.

References

Lostings. (n.d.). Retrieved from Lostings: https://www.lostings.com

LostnFound. (n.d.). Retrieved from https://www.lostnfoundpak.com/

Zainab Alert App. (n.d.).

Zara Alert App. (n.d.).

Appendix A

Use Case Description Template

UC-1: User Registration

Use Case ID	UC-1
Use Case Name	User Registration
Actors	Unregistered User
Description	This use case allows a new user to create an account in the system.
Trigger	User clicks on "Sign Up" button
Preconditions	User must not already be registered
Postconditions	User account is created successfully
Normal Flow	1. User opens system 2. Clicks on Sign Up 3. Enters name, email, password, phone 4. System validates data 5. Account is created
Alternative Flows	Email already exists → show error Invalid data → ask user to correct
Business Rules	Email must be unique and valid
Assumptions	User has internet connection

UC-2: User Login

Use Case ID	UC-2
Use Case Name	User Login
Actors	Registered User
Description	This use case allows a user to login into the system
Trigger	User clicks on "Login"
Preconditions	User must be registered
Postconditions	User is redirected to dashboard

Normal Flow	1. User enters email and password 2. System verifies credentials 3. User logged in successfully
Alternative Flows	Wrong password → error message User not found
Business Rules	Authentication must be secure
Assumptions	System server is active

UC-3: Create Report

Use Case ID	UC-3
Use Case Name	Create Report (Lost/Found)
Actors	Registered User
Description	User can create a report for lost or found item/person
Trigger	User clicks "Create Report"
Preconditions	User must be logged in
Postconditions	Report is saved in database
Normal Flow	1. User opens report form 2. Enters title, description, location etc. 3. Uploads image 4. Submits report
Alternative Flows	Missing fields → show error Image upload fails
Business Rules	Report must contain required details
Assumptions	User provides correct information

UC-4: Search & Filter Reports

Use Case ID	UC-4
Use Case Name	Search and Filter Reports
Actors	All Users
Description	Users can search and filter reports
Trigger	User enters search query
Preconditions	Reports must exist in system
Postconditions	Filtered results are shown
Normal Flow	1. User enters keyword 2. Applies filters (location/type/name) 3. System displays results
Alternative Flows	No results found
Business Rules	Search must be fast and accurate
Assumptions	Database is available

UC-4: Comment on report

Use Case ID	UC-5
Use Case Name	Comment on Report
Actors	Registered User
Description	User can comment on reports
Trigger	User clicks "Comment"
Preconditions	User must be logged in
Postconditions	Comment is added to report

Normal Flow	1. User writes comment 2. Clicks submit 3. Comment saved
Alternative Flows	Empty comment → error
Business Rules	Comment must not be empty
Assumptions	User behaves appropriately

UC-6: View Statistics

Use Case ID	UC-6
Use Case Name	View Statistics
Actors	All Users
Description	User views system statistics.
Trigger	Open stats page
Preconditions	Data exists
Postconditions	Stats displayed
Normal Flow	1. Open page 2. View stats
Alternative Flows	No data
Business Rules	Data must be correct
Assumptions	System active

UC-7: View Posts

Use Case ID	UC-7
Use Case Name	View Posts

Actors	All Users
Description	User views posts in system.
Trigger	Open homepage
Preconditions	Posts exist
Postconditions	Posts displayed
Normal Flow	1. Open system 2. View posts
Alternative Flows	No posts
Business Rules	Only valid posts shown
Assumptions	Data available

UC-8: Share Posts

Use Case ID	UC-8
Use Case Name	Share Posts
Actors	All Users
Description	User shares posts with others.
Trigger	Click share
Preconditions	Post exists
Postconditions	Post shared
Normal Flow	1. Select post 2. Click share
Alternative Flows	Share fails
Business Rules	Share allowed
Assumptions	Internet available

UC-9: View Comments

Use Case ID	UC-9
Use Case Name	View Comments
Actors	All Users
Description	User views comments on posts.
Trigger	Open comments
Preconditions	Comments exist
Postconditions	Comments shown
Normal Flow	1. Open post 2. View comments
Alternative Flows	No comments
Business Rules	Comments visible
Assumptions	Data available

UC-10: Edit Reports

Use Case ID	UC-10
Use Case Name	Edit Report
Actors	Registered User
Description	User edits report.
Trigger	Click edit
Preconditions	Owner access
Postconditions	Updated report

Normal Flow	1. Edit 2. Save
Alternative Flows	Unauthorized
Business Rules	Only owner
Assumptions	Logged in

UC-11: Delete Report

Use Case ID	UC-11
Use Case Name	Delete Report
Actors	Registered User
Description	User deletes report.
Trigger	Click delete
Preconditions	Owner
Postconditions	Deleted
Normal Flow	1. Select delete 2. Confirm
Alternative Flows	Cancel
Business Rules	Only owner delete
Assumptions	User confirmed

UC-12: Flag Content

Use Case ID	UC-12
Use Case Name	Flag Content
Actors	Registered User

Description	User flags inappropriate content.
Trigger	Click flag option
Preconditions	Logged in
Postconditions	Content flagged
Normal Flow	1. Select post 2. Click flag 3. Submit
Alternative Flows	Duplicate flag
Business Rules	Valid reason required
Assumptions	Honest reporting

UC-13: Reset Password

Use Case ID	UC-13
Use Case Name	Reset Password
Actors	Registered User
Description	User resets password using email.
Trigger	Click forgot password
Preconditions	User registered
Postconditions	Password updated
Normal Flow	1. Enter email 2. Receive link 3. Set new password
Alternative Flows	Invalid email

Business Rules	Link expires
Assumptions	Email service working

UC-14: Log Out

Use Case ID	UC-14
Use Case Name	Log Out
Actors	Registered User
Description	User logs out from system.
Trigger	Click logout
Preconditions	User logged in
Postconditions	Session ended
Normal Flow	1. Click logout 2. Session cleared
Alternative Flows	Session expired
Business Rules	Session must end
Assumptions	User active

UC-15: Manage Users

Use Case ID	UC-15
Use Case Name	Manage Users
Actors	Admin
Description	Admin manages user accounts.
Trigger	Admin opens dashboard
Preconditions	Admin logged in

Postconditions	Users managed
Normal Flow	1. View users 2. Add/update/delete
Alternative Flows	Invalid action
Business Rules	Only admin allowed
Assumptions	Admin authorized

UC-16: Verify Cases by Tracking

Use Case ID	UC-16
Use Case Name	Verify Cases by Tracking
Actors	Admin
Description	Admin verifies cases using details.
Trigger	Admin selects case
Preconditions	Case exists
Postconditions	Case verified
Normal Flow	1. Open case 2. Check 3. Verify
Alternative Flows	Invalid data
Business Rules	Only verified cases shown
Assumptions	Accurate data

UC-17: Update Case Status

Use Case ID	UC-17
Use Case Name	Update Case Status
Actors	Admin
Description	Admin updates case status.
Trigger	Select case
Preconditions	Case exists
Postconditions	Status updated
Normal Flow	1. Select case 2. Update status 3. Save
Alternative Flows	Invalid status
Business Rules	Status must be valid
Assumptions	Admin active

UC-18: Manage Posts

Use Case ID	UC-18
Use Case Name	Manage Posts
Actors	Admin
Description	Admin manages system posts.
Trigger	Admin selects post
Preconditions	Posts exist
Postconditions	Post updated/deleted

Normal Flow	1. View posts 2. delete flagged or inappropriate contents
Alternative Flows	Unauthorized
Business Rules	Admin control only
Assumptions	Admin logged in

UC-19: Send Alerts to Users

Use Case ID	UC-19
Use Case Name	Send Alerts to Users
Actors	Admin
Description	Admin sends alerts/notifications to users.
Trigger	Admin sends alert
Preconditions	Users exist
Postconditions	Alert delivered
Normal Flow	1. Create alert 2. Send to users
Alternative Flows	Delivery fails
Business Rules	Correct users notified
Assumptions	Notification system working

Appendix-B

Coding Standards

General Coding Standards & Guidelines

1.Consistent Variable Naming

We use camel case for variables and pascal case for classes.

2. Meaningful Naming

Variables and fields are named clearly to describe their purpose.

3.Immutable Fields

Fields are kept constant after initialization using const or read only where required.

4.Controlled Access

Sensitive data is accessed using get only properties to prevent unauthorized updates.

5.Function Naming

Functions are named based on their functionality.

6.Comments Usage

Comments are added for feedback and for user while inappropriate comments can be removed or hide by admin.

7.Indentation & Formatting

Proper indentation is used to clearly define loops, conditions, and blocks for better readability.

8.File & Folder Naming

Files and folders are named according to their content.

9.Class Structure

Each class follows a clear structure with properties, constructors and methods

10. Code Organization

Application is divided into frontend, backend and database. Each class is stored in a separate file.

11.Minimum Scope Usage

Variables are declared only where needed to avoid unnecessary access.

12.Access Modifiers

Proper use of public and private is followed to secure data and restrict access.

13.High Cohesion & Low Coupling

Each module performs a specific task and is connected to others.

14.DRY Principle

i. Reusable functions are created for repeated tasks.
ii. Matching logic is written once and reused.

iii. Methods are kept short and simple.
iv. Avoid long and complex functions.

15.Single Responsibility Principle
Each class handles only one responsibility

16.Extenable Code
Classes and functions are written in a way that they can be extended without modifying existing code.

17.Exception Handling
Errors are handled using try-catch blocks.

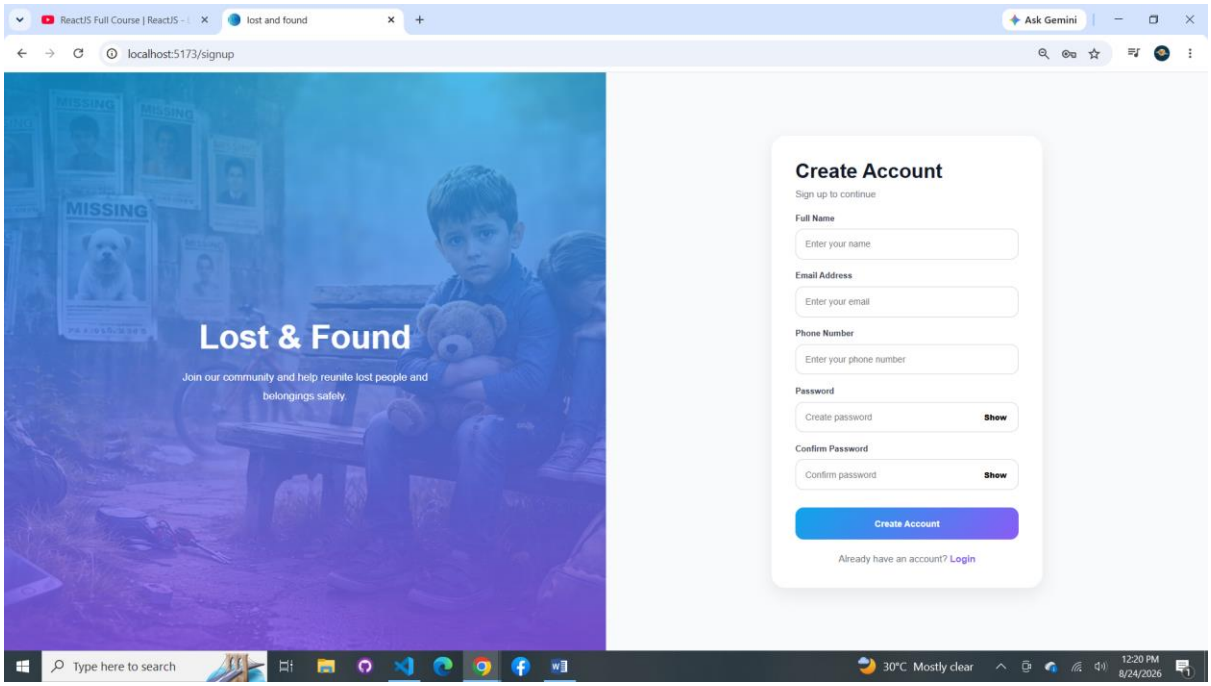
18.Logging
Important events like report submission, verification and errors are logged for tracking.

19.Consistent Logging Style
A uniform logging format is used across the system.

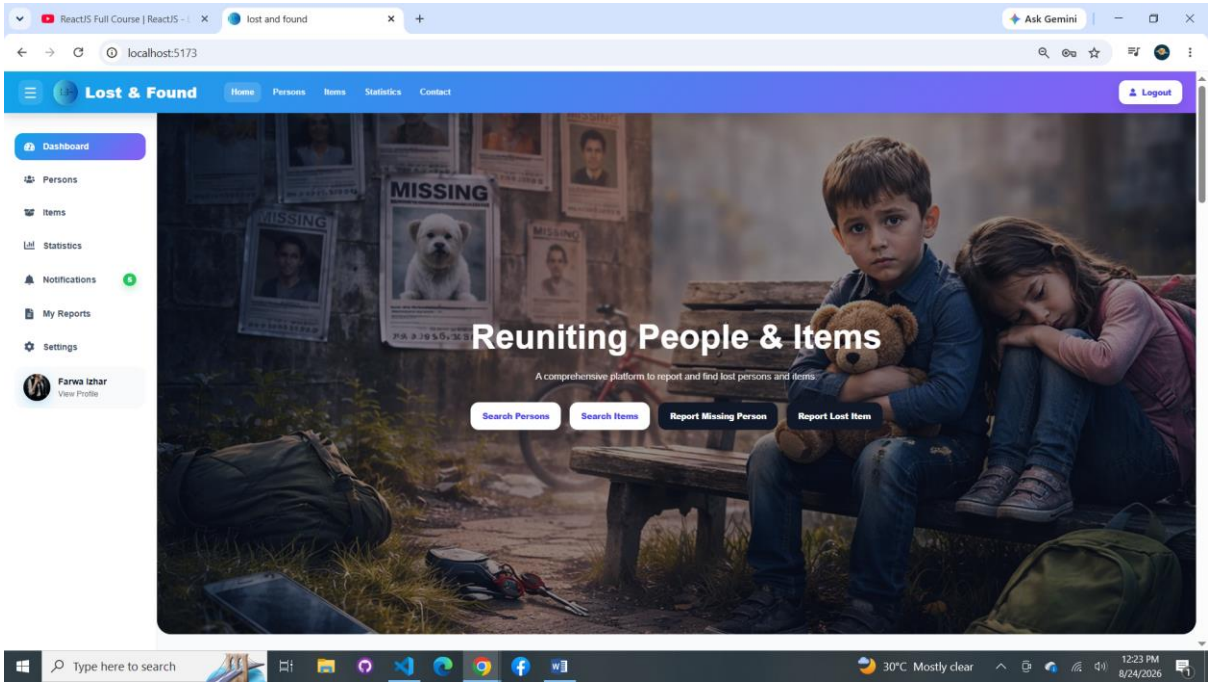
Appendix C

Prototypes

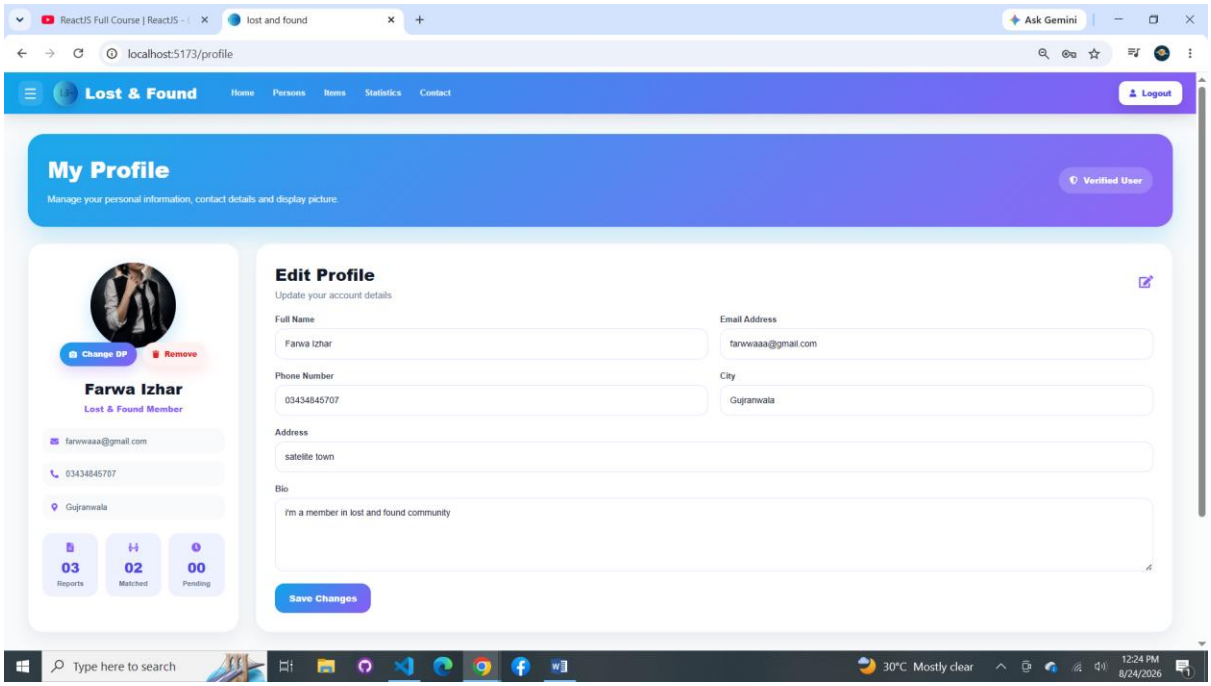
Sign up Page



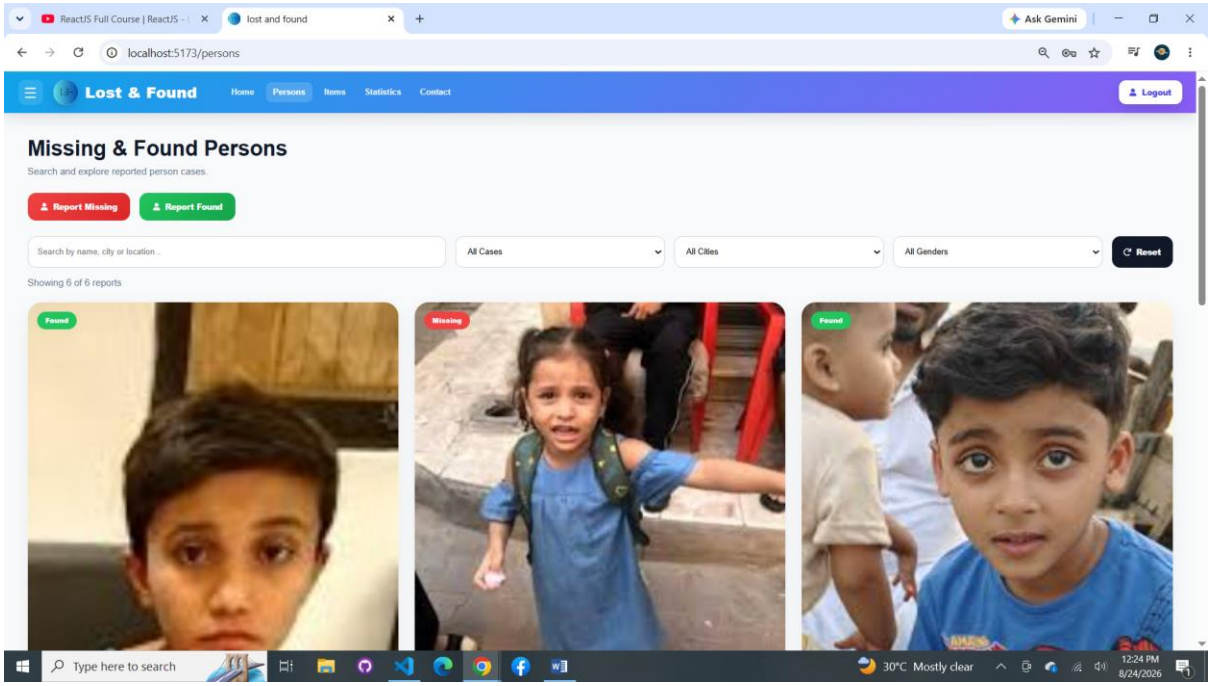
Home Page



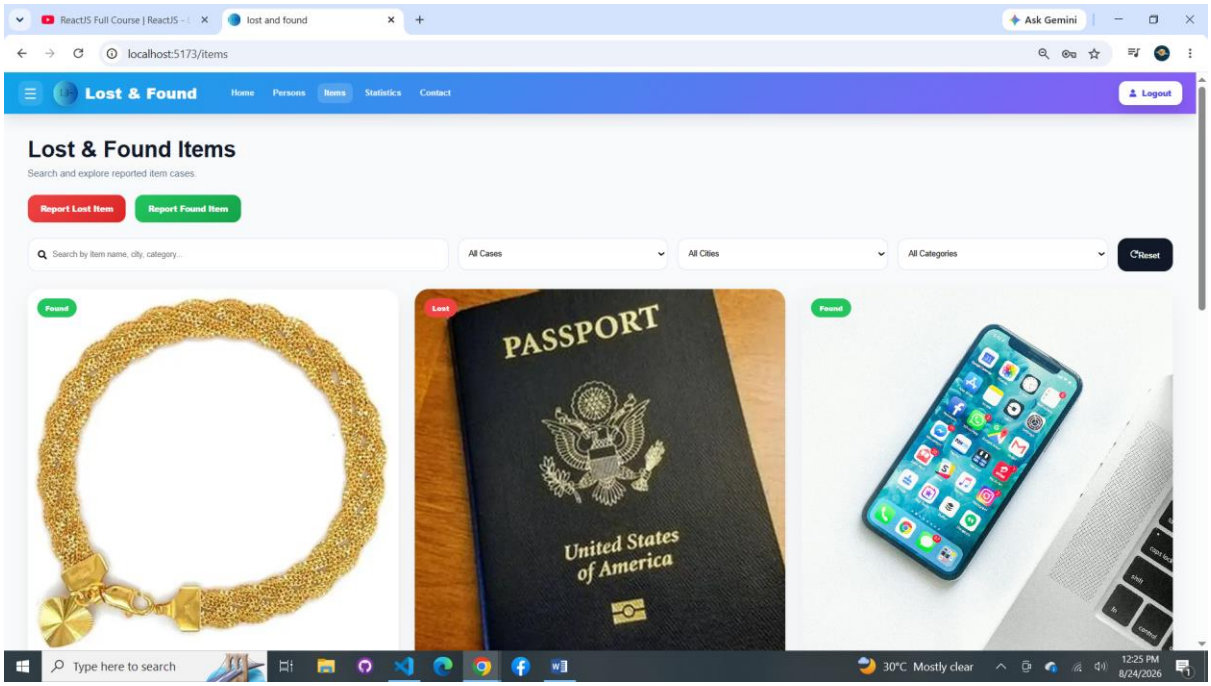
Profile



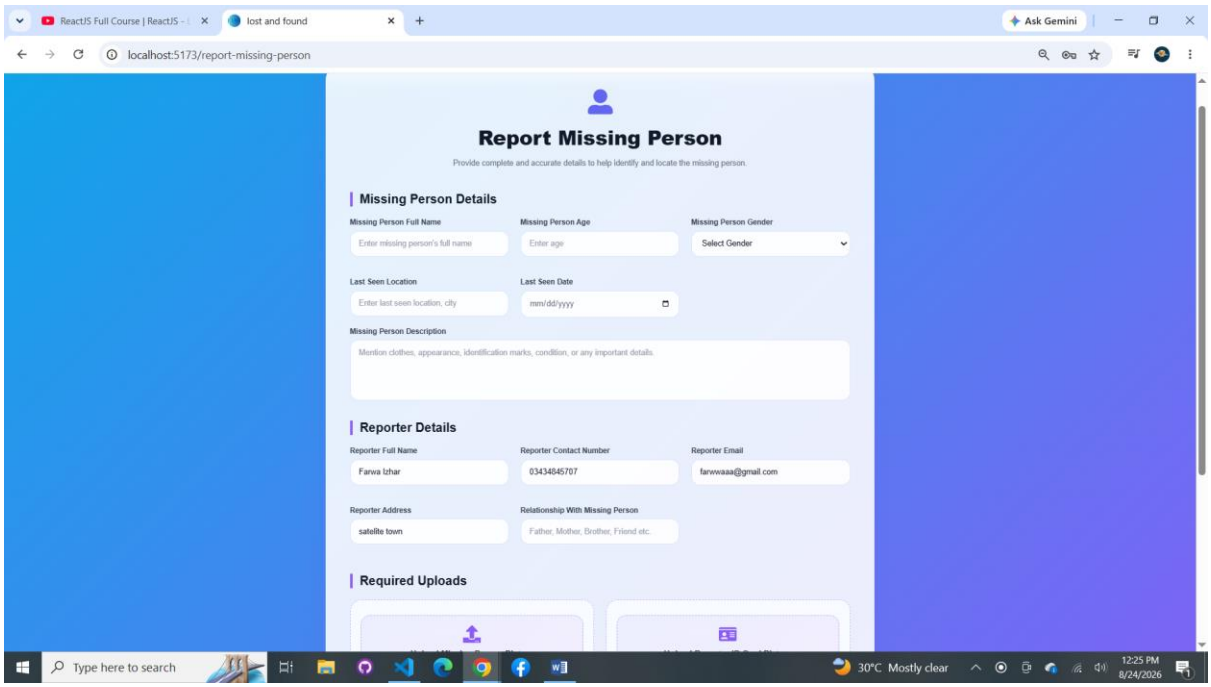
Persons



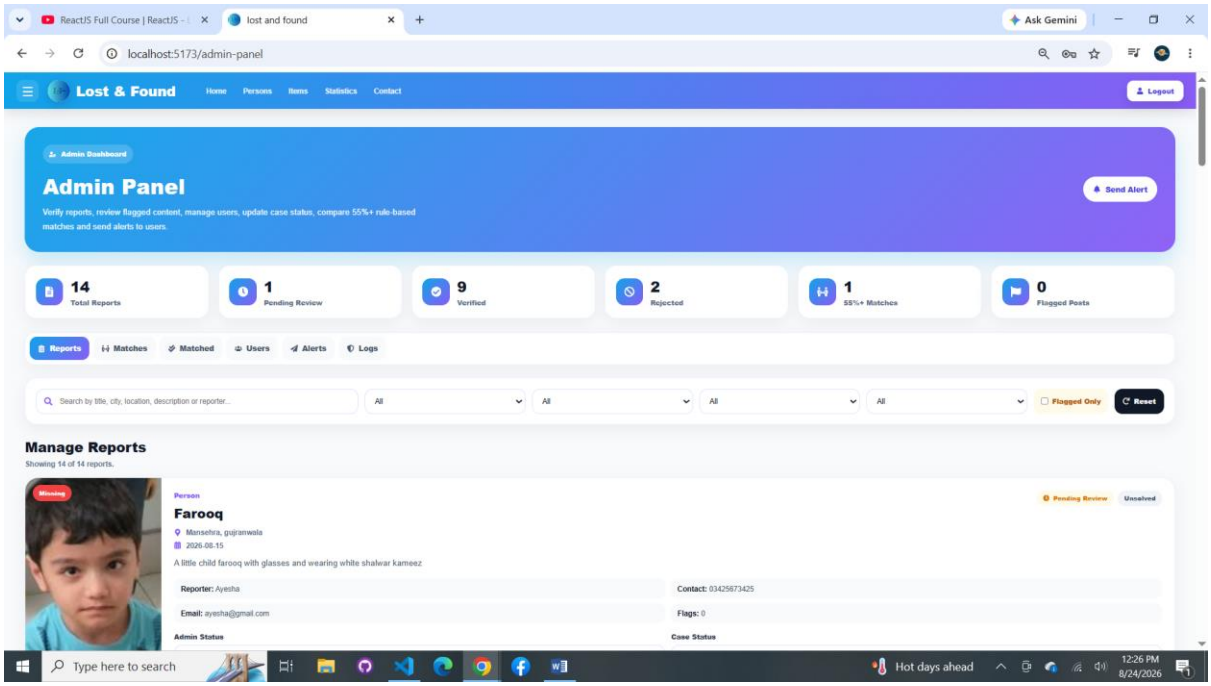
Items


lost and found
localhost:5173/items
Lost & Found
Home
Persons
Items
Statistics
Contact
Logout
Lost & Found Items
Search and explore reported item cases.
Report Lost Item
Report Found Item
Search by item name, city, category...
All Cases
All Cities
All Categories
Reset
Found
Lost
PASSPORT
United States of America
Found
Type here to search
30°C Mostly clear
12:25 PM
8/24/2026

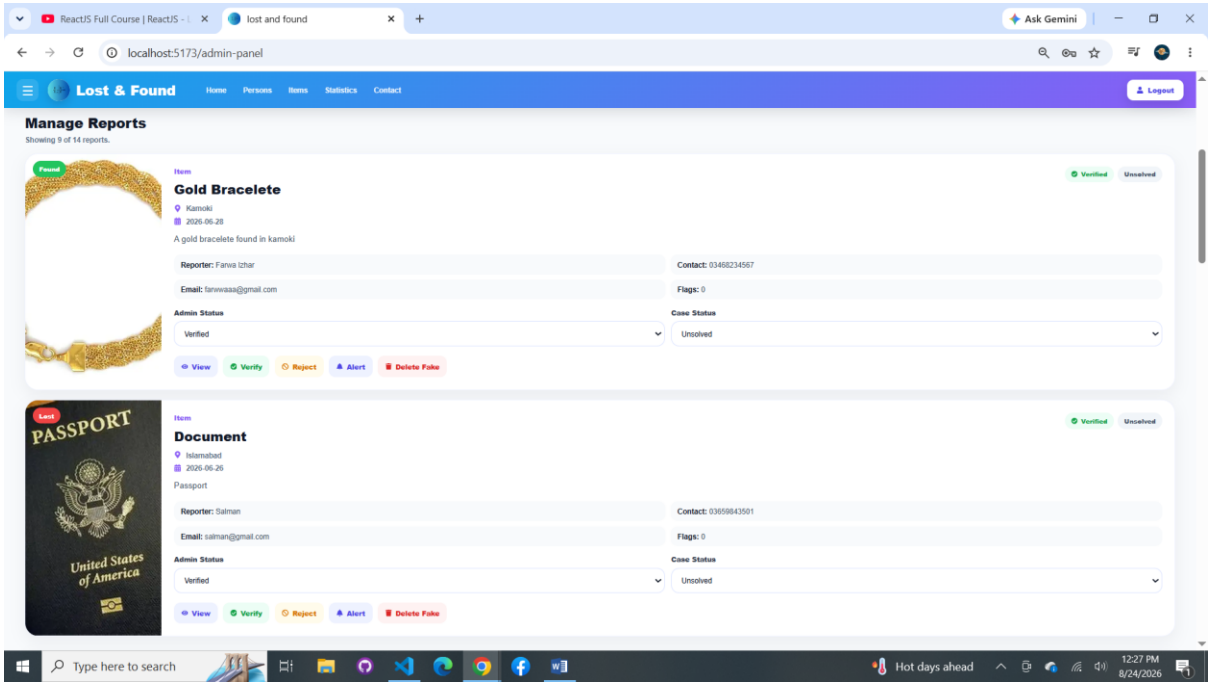
Form


lost and found
localhost:5173/report-missing-person
Report Missing Person
Provide complete and accurate details to help identify and locate the missing person.
Missing Person Details
Missing Person Full Name
Enter missing person's full name
Missing Person Age
Enter age
Missing Person Gender
Select Gender
Last Seen Location
Enter last seen location, city
Last Seen Date
mm/dd/yyyy
Missing Person Description
Mention clothes, appearance, identification marks, condition, or any important details.
Reporter Details
Reporter Full Name
Farwa Izhar
Reporter Contact Number
03434845707
Reporter Email
farwwaaa@gmail.com
Reporter Address
satellite town
Relationship With Missing Person
Father, Mother, Brother, Friend etc.
Required Uploads
Type here to search
30°C Mostly clear
12:25 PM
8/24/2026

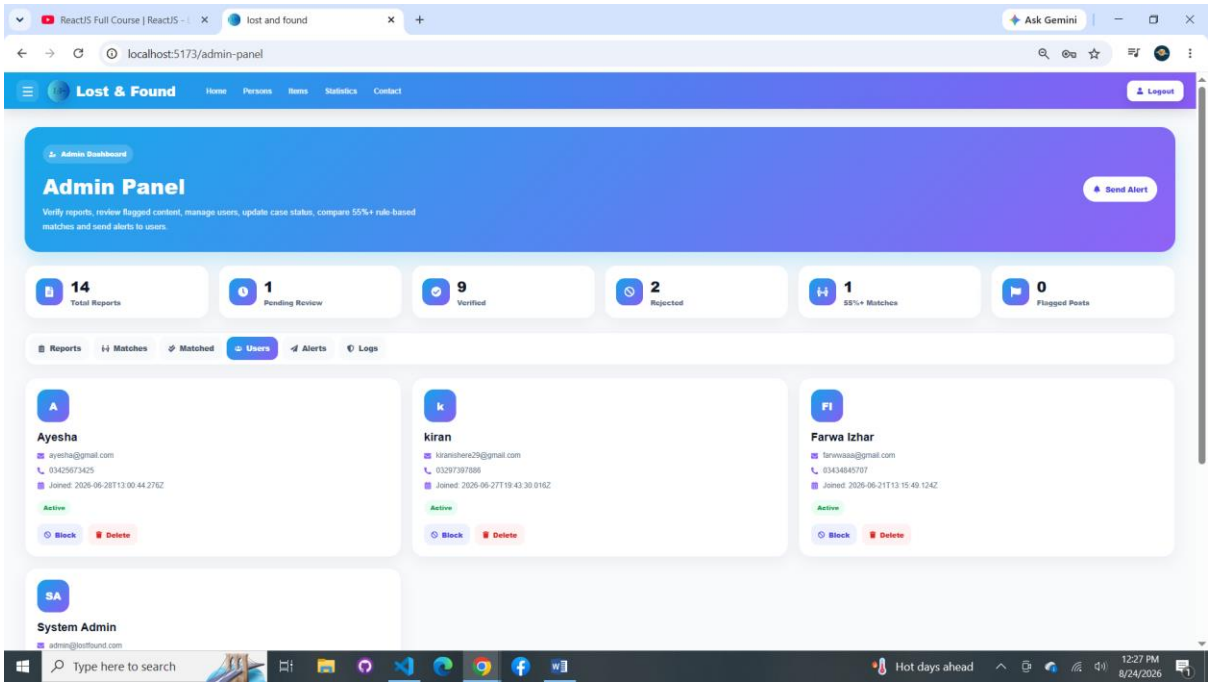
Admin Dashboard



Verify Cases



Manage Users



Rule-based matches

